Abschlussprüfung “Angular” NE4NE8

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur Kurs-Projektarbeit

Earth Oceans Learning APP

Entwicklung einer kindgerechten Angular SPA zur Vermittlung von Wissen über die 5 Ozeane für Kinder (7-9 Jahre)

Abgabe der Dokumentation: 09.12.2025

Prüfling:

Tobias Boyke

Musterstraße 1

40470 Düsseldorf

Prüfungsnummer: 0001278495

Ausbildungsbetrieb:

Beispiel GmbH

Firmenweg 2

41460 Neuss

Inhaltsverzeichnis

1. Einleitung **5**
1.1. Ausgangssituation 5
1.2. Projektidee und Zielsetzung 5
1.3 Projektbegründung 5
1.4 Make-or-Buy Entscheidung 6
2. Projektplanung **7**
2.1 Ist-Analyse 7
2.2. Soll-Analyse 7
2.3 Zeitplanung 8
2.4 Kostenplanung 10
3. Analyse & Entwurf **10**
3.1 Anwendungsfalldiagramm (Use Cases) 10
3.2 Architekturentwurf 12
3.3 UI/UX Design 14
3.4 Datenmodell 15
3.5 Klassendiagramm (UML) 16
3.6 Datenschutz & Sicherheit (Privacy by Design) 16
4. Realisierung **17**
4.1 Entwicklungsumgebung 17
4.2 Implementierung der Hauptkomponenten 18
4.2.1 Standalone Components 18
4.2.2 State Management via QuizStore 19
4.2.3 Routing & Navigation 20
4.2.4 CSS-Architektur und Styling-Strategie 21
4.3 Entwicklung des Backend-Moduls 23
4.4 Herausforderungen & Lösungen 24
5. Qualitätssicherung **25**
5.1 Automatisierte Qualitätssicherung mittels CI/CD 26
5.2 Google Lighthouse Audit 27
6. Wirtschaftlichkeitsbetrachtung **28**
6.1 Soll-Ist-Vergleich (Zeit) 28
6.2 Nachkalkulation (Kosten) 28
6.3 Amortisationsrechnung 29
7. Fazit & Ausblick **30**
7.1 Zusammenfassung 30
7.2 Lessons Learned 30
7.3 Ausblick 33
7.4 Projektabnahme und Übergabe 33
A. Anhang **34**
A.0 Quellen- und Literaturverzeichnis 34
A.1 Verzeichnisstruktur 35
A.2. Quellcode und Dateien 35
A.3. Mockups und Screenshots 36
A.4 Erweiterung der FCPP Umsetzungs-Elemente 38
A.5 Code-Beispiele 40
A.6 Benutzerhandbuch 46
A.7 Abnahmeprotokoll 47
A.8 Erklärung **48**
A.9 Realer Zeitaufwand & Reflexion 48

Abbildungsverzeichnis

2.3 Zeitplanung - Gantt Diagramm ……07

3.1 Anwendungsfalldiagramm (Use Cases)....09

3.2 Architekturentwurf.... 10

3.3 UI/UX Design……..11

3.4 Datenmodell....13

3.5 Klassendiagramm (UML).... 14

Tabellenverzeichnis

2.3 Zeitplanung.... 08

4.3 Herausforderungen & Lösungen....20

5.2 Testdurchführung & Ergebnisse....22

6.1 Soll-Ist-Vergleich (Zeit)....24

Listings

3.4 Datenmodell ……13

4.2.1 Standalone Components ……...15

4.2.2 State Management im QuizStore ……16

4.2.3 Routing & Navigation ……17

4.2.4 CSS-Architektur und Styling-Strategie……18-19

4.3 Failsafe handling……23

5.1 Testplanung ……21

Glossar

Begriff	Erklärung
Angular	Ein von Google entwickeltes, TypeScript-basiertes Open-Source-Framework zur Erstellung von skalierbaren Webanwendungen[1].
Angular Signals	Ein reaktives Primitiv in Angular, das es ermöglicht, Zustandsänderungen feingranular zu verfolgen. Signals informieren Templates und Abhängigkeiten direkt über Änderungen, was die Performance verbessert und die Abhängigkeit von Zone.js reduziert [2]
BEM	*Block-Element-Modifier* - Eine Namenskonvention für CSS-Klassen, die hilft, den Code modular, wiederverwendbar und strukturiert zu halten, indem sie die Abhängigkeit zwischen HTML und CSS explizit macht.[8]
CI/CD	*Continuous Integration / Continuous Delivery* - Eine Methode der Softwareentwicklung, bei der Code-Änderungen automatisch getestet und bereitgestellt werden.
CMS	*Content Management System* - Software zur gemeinschaftlichen Erstellung und Organisation von Inhalten (z.B. GRAV, WordPress).
CSR	*Client-Side Rendering* - Verfahren, bei dem die Darstellung der Webseite im Browser (Client) mittels JavaScript erfolgt, anstatt fertig gerenderte HTML-Seiten vom Server zu laden. Dies entlastet den Server und ermöglicht ein "App-Feeling".
FCPP	*Fast Concept Projekt Plan, eine Idee von Herrn Ehlen*
JSON	*JavaScript Object Notation* - Ein kompaktes, textbasiertes Datenformat zum Datenaustausch, das für Menschen einfach zu lesen ist.
Kanban	*Eine Methode der agilen Prozesssteuerung, bei der Aufgaben auf einem Board (hier: GitHub Projects) visualisiert werden, um den Arbeitsfluss zu optimieren und die Menge paralleler Arbeit ("Work in Progress") zu begrenzen.*
LCP	*Largest Contentful Paint – Eine Metrik von Google, die misst, wie lange es dauert, bis das größte sichtbare Inhaltselement geladen ist*
LocalStorage	Teil der Web Storage API, der es ermöglicht, Daten (Key-Value-Paare) dauerhaft im Browser des Benutzers zu speichern.[4]
MVP	Minimum Viable Product – Ein minimal funktionsfähiges Produkt, das nur die nötigsten Kernfunktionen enthält.
MVVM	*Model-View-ViewModel* - Ein Architekturmuster, das die grafische Benutzeroberfläche von der Geschäftslogik trennt.
Playwright	*Ein modernes Framework für End-to-End-Tests, das zuverlässige Automatisierung in verschiedenen Browsern (Chromium, Firefox, WebKit) ermöglicht*
SPA	*Single Page Application* - Eine Webanwendung, die aus einer einzigen HTML-Seite besteht und Inhalte dynamisch nachlädt.
Standalone Component	Ein Konzept in Angular, bei dem Komponenten ohne NgModules auskommen, was die Struktur vereinfacht.[1]
Tailwind CSS 4	Die neuste Version des CSS-Frameworks, die auf eine Konfigurationsdatei verzichtet und durch eine reine CSS-Konfiguration (@theme) performanter ist.
TypeScript	Eine Programmiersprache von Microsoft, die JavaScript um statische Typisierung erweitert.
Vitest	Ein extrem schnelles Unit-Test-Framework, das speziell für moderne Webprojekte (Vite-basiert) entwickelt wurde und oft Jest ersetzt.
WCAG	*Web Content Accessibility Guidelines* - Internationaler Standard zur barrierefreien Gestaltung von Internetangeboten.[7]

1. Einleitung

1.1. Ausgangssituation

Im Rahmen der Umschulung zum Fachinformatiker für Anwendungsentwicklung, erläutert der Autor, welcher bei Herrn Ehlen sein Kurs-Element “Angular” absolvierte, den Ablauf seines hier folgenden Abschlussprojekts **bei einem fiktiven** Unternehmen.

Die **Beispiel GmbH** ist ein junges, aufstrebendes IT-Unternehmen mit Sitz im Herzen von Neuss. Seit ihrer Gründung im Jahr 2025 hat sich die Firma darauf spezialisiert, maßgeschneiderte Webanwendungen und anspruchsvolle CMS-Lösungen für Unternehmenskunden zu entwickeln. **14 Mitarbeiter**, bestehend aus Backend- und Frontend-Entwicklern, UI/UX-Designern, Projektmanagern und Content-Spezialisten, sind in der Beispiel GmbH beschäftigt und arbeiten vorwiegend für Kunden aus dem produzierenden Gewerbe, der Logistik und der Edutainment-Branche.

Die Beispiel GmbH wurde von einem Kunden, einer Umweltschutz-NGO, beauftragt, eine Browser Applikation für das spielerische Erlernen von Ozean-Fakten für Grundschulkinder zu entwickeln.

Zu den Stakeholdern des Projekts zählen:

- Der Auftraggeber (NGO): Vertreten durch Herrn Dr. Uwe Umwelt.

1.2. Projektidee und Zielsetzung

Ziel des Projektes ist die Entwicklung einer Single Page Application (SPA) "Earth Ocean Learning". Die Anwendung soll Kindern spielerisch Wissen über die fünf Ozeane vermitteln. Kernfunktionen umfassen eine Auswahl der Ozeane, einen Lernbereich mit Informationen sowie ein Quiz-Modul zur Wissensüberprüfung.Die Anwendung muss als eigenständiges Modul (Standalone) konzipiert sein, um eine einfache Integration in bestehende CMS-Systeme (wie GRAV) zu ermöglichen.

1.3 Projektbegründung

Die Digitalisierung im Bildungssektor erfordert moderne, webbasierte Lernlösungen, die plattformunabhängig funktionieren. Herkömmliche Lernmaterialien (PDFs, statische Webseiten) bieten oft nicht den nötigen Anreiz für die Zielgruppe (Kinder). Durch den Einsatz moderner Webtechnologien (Angular, Signals) soll eine performante, wartbare und zukunftssichere Lösung geschaffen werden, die gleichzeitig als Referenzprojekt für die Beispiel GmbH dient, um die Kompetenz im Bereich "Edutainment" zu unterstreichen.

1.4 Make-or-Buy Entscheidung

Im Vorfeld des Projekts wurde geprüft, ob der Einsatz von Standard-Lösungen (SaaS-Produkte wie Kahoot!, Quizlet oder Moodle[1]) möglich ist (Buy). Die Entscheidung fiel zugunsten einer Eigenentwicklung (Make) aus, da dies eine explizite Vorgabe des Auftraggebers (NGO) war.

Diese Vorgabe wurde durch eine technische Analyse der Beispiel GmbH bestätigt und als wirtschaftlich sinnvoll validiert:

Datenschutz: Da die Zielgruppe Kinder (7–9 Jahre) umfasst, forderte die NGO eine Lösung ohne Datenerhebung (No-Tracking) und ohne Benutzerkonten-Zwang. Gängige Markt-Lösungen sammeln oft Metadaten oder erfordern Logins, was den strengen Datenschutzanforderungen der NGO widersprach.

Offline-Fähigkeit & Integration: Die Anwendung muss langfristig nahtlos in das bestehende CMS der NGO integrierbar sein und perspektivisch auch offline auf Tablets in Schulen (ohne stabiles WLAN) funktionieren. Eine PWA-fähige Eigenentwicklung auf Basis von Angular erfüllt diese Anforderung besser als webbasierte Abo-Modelle.

Lizenzkosten vs. Einmalkosten: Bei der geplanten hohen Nutzerzahl und der Laufzeit über mehrere Jahre sind die einmaligen Entwicklungskosten ("Make") günstiger als die laufenden, nutzerbasierten Lizenzgebühren ("Buy") kommerzieller Bildungsplattformen.

Hybrid-Ansatz (Eigenentwicklung + CMS Integration): Die Entscheidung zur Eigenentwicklung erlaubt eine flexible "Hybrid-Architektur**"**. Die Anwendung wurde so konzipiert, dass sie sowohl Standalone als auch Connected funktioniert.
Dies wäre mit geschlossenen "Buy"-Lösungen nicht möglich gewesen und bietet der NGO maximale Flexibilität für zukünftige Deployments.

[1] Software as a Service – Ein Lizenz- und Vertriebsmodell, bei dem Software als Dienstleistung über das Internet bereitgestellt wird.

2. Projektplanung

2.1 Ist-Analyse

Zum Zeitpunkt des Projektstarts existiert keine Softwarelösung für den genannten Anwendungsfall. Die Inhalte (Texte, Bilder zu Ozeanen) liegen lediglich in unstrukturierter Form (Textdokumente, Bilddateien) vor. Es gibt keinen bestehenden Codebase, auf dem aufgebaut werden kann ("Greenfield Project").

Technische Ausgangslage:

- Entwickler-Workstation mit Windows 11 und Rocky Linux 10.
- Zugriff auf Standard-Entwicklungstools (VS Code, Sublime Text 4 (Build 4200), GitHub, Node.js, NPM.
- Eine bestehende CI/CD Pipeline für dieses spezifische Projekt.

2.2. Soll-Analyse

Es soll eine Webanwendung entwickelt werden, die folgende Anforderungen erfüllt:

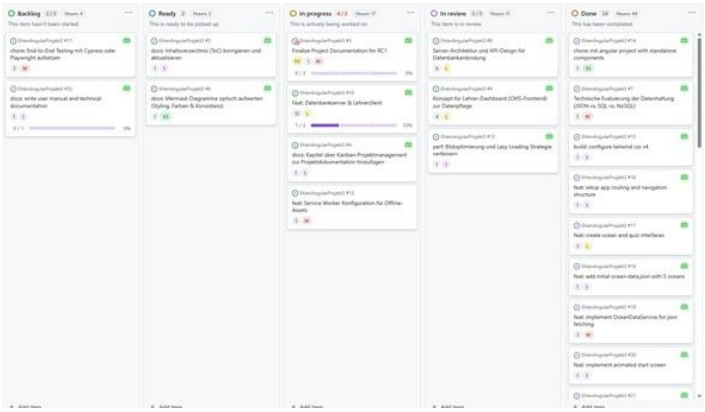
Funktionale Anforderungen:

- **Ozean-Auswahl:** Visuelle Darstellung und Auswahl der 5 Weltmeere.
- **Lernmodus:** Anzeige von Fakten und Bewohnern pro Ozean (Carousel/Slider).
- **Quiz-System:** Multiple-Choice Fragen mit direktem Feedback.
- **Fortschritt:** Speicherung des Lernfortschritts (Sterne-System) im lokalen Browser-Storage (kein Backend-Zwang).
- **Master-Quiz:** Ein freischaltbarer Modus nach Abschluss aller Ozeane.

Nicht-funktionale Anforderungen:

- **Zielgruppe:** Kindgerechtes UI/UX (große Buttons, wenig Text, viel Bild).
- **Technologie:** Angular 21 (aktuellste Version), Nutzung von Standalone Components.
- **State Management:** Nutzung von Angular Signals für reaktives Datenmanagement.
- **Performance:** Kurze Ladezeiten, optimierte Assets.
- **Responsive Design:** Lauffähig auf Tablets und Desktops.

Methodisches Vorgehen: Agilisierung des FCPP mittels Kanban Um den im FCPP definierten Projektrahmen flexibel und effizient umzusetzen, wurde die **Kanban-Methodik** gewählt. Der initiale FCPP-Entwurf diente dabei als Basis (Backlog), wurde jedoch zur operativen Steuerung in **GitHub Projects** in dynamische Arbeitspakete (Issues) überführt. Durch die Visualisierung des Arbeitsflusses auf dem Kanban-Board (*Todo, In Progress, Done*) konnte der statische Plan agil an die technischen Realitäten – wie etwa die kurzfristigen Anpassungen im Testing-Bereich – angepasst werden. Dies verhinderte Stillstände und sorgte für einen kontinuierlichen Entwicklungsfortschritt ("Flow").



2.3 Zeitplanung

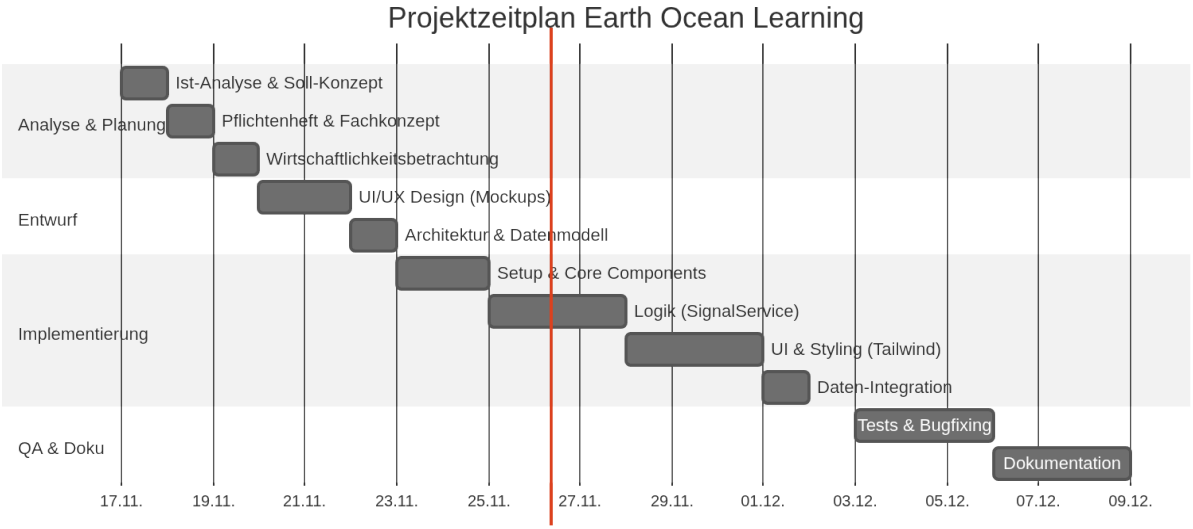


Abb 2.3 Visualisierung der Zeitplanung mittels Gantt-Diagramm
Der Durchführungszeitraum des Projektes ist vom 17.11.2025 bis 09.12.2025.

Phase	Tätigkeit	Geplante Zeit (h)
1. Analyse & Planung		*11 h*
	Ist-Analyse & Soll-Konzept	*3 h*
	Erstellung Pflichtenheft / Fachkonzept	*4 h*
	Wirtschaftlichkeitsbetrachtung (Planung)	*2 h*
	Zeit- & Ressourcenplanung	*2 h*
2. Entwurf		*12 h*
	UI/UX Design (Mockups)	*5 h*
	Software-Architektur & Datenmodellierung	*4 h*
	Auswahl der Bibliotheken & Tools	*3 h*
3. Implementierung		*28 h*
	Aufsetzen der Entwicklungsumgebung	*2 h*
	Implementierung Core-Komponenten & Routing	*6 h*
	Implementierung Logik (QuizStore, Services)	*8 h*
	Implementierung UI & Styling (Tailwind)	*8 h*
	Integration der Daten (JSON)	*4 h*
4. Qualitätssicherung		*9 h*
	Erstellung von Testfällen	*3 h*
	Durchführung Tests & Bugfixing	*6 h*
5. Dokumentation		*10 h*
	Erstellung Projektdokumentation	*8 h*
	Erstellung Benutzerhandbuch	*2 h*
Gesamt		***70 h***

Tab.2.3. Die geplante Gesamtzeit beträgt 70 Stunden.

2.4 Kostenplanung

Die Kostenplanung basiert auf den internen Kostensätzen für Umschüler des Bildungsträgers bzw. des Praktikumsbetriebs. Da keine direkten Lohnkosten anfallen, wird der Kostensatz der Umschulungsmaßnahme als Berechnungsgrundlage herangezogen.

Personalkosten:

- Interner Verrechnungssatz (Umschüler): 9,00 €
- Geplante Stunden: 70 h
- Summe Personal: 70 h * 9,00 €/h = 630,00 €

Sachmittelkosten:

- Nutzung Entwicklungsumgebung (Pauschale für Arbeitsplatz/Strom): 150,00 €
- Summe Sachmittel: 150,00 €

Gesamtkosten (Plan): 780,00 €

3. Analyse & Entwurf

3.1 Anwendungsfalldiagramm (Use Cases)

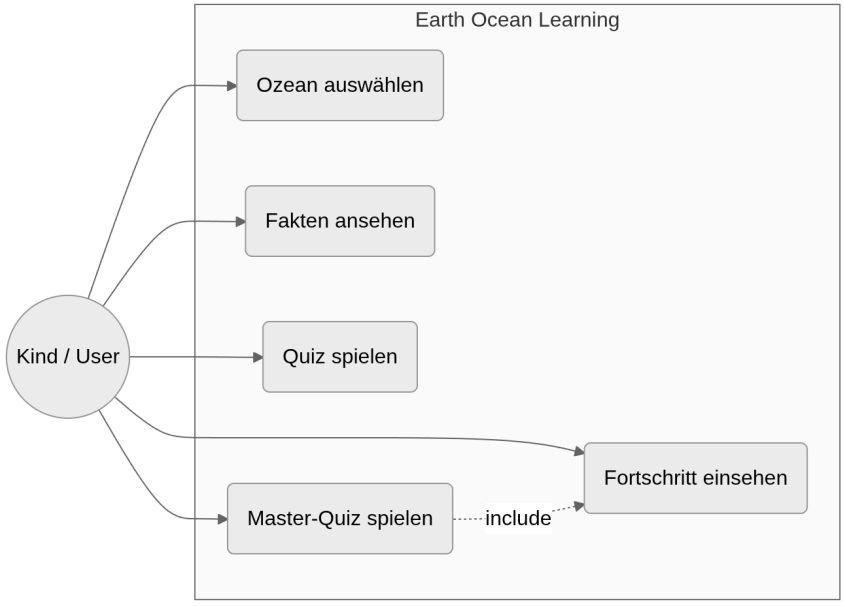


Abb.3.1 UML-Anwendungsfalldiagramm des Systems „Earth Ocean Learning“ mit den zentralen Funktionen für den Akteur „Kind / User“.

3.2 Architekturentwurf

Die Anwendung wurde als reine Single Page Application (SPA) konzipiert, die vollständig im Browser des Clients ausgeführt wird (Client-Side Rendering).

Im Gegensatz zu klassischen Webseiten, die bei jeder Interaktion neu vom Server geladen werden, lädt diese SPA lediglich einmalig die notwendigen Ressourcen (HTML, CSS, JavaScript) und tauscht anschließend Inhalte dynamisch aus. Auf Server-Side Rendering (SSR) wurde nach einer Evaluierung bewusst verzichtet. Da es sich um eine geschlossene Lernanwendung handelt, sind SEO-Optimierungen (Suchmaschinenranking) zweitrangig. Der Verzicht auf SSR reduziert die Komplexität der Infrastruktur drastisch (kein Node.js-Server für den Betrieb nötig) und ermöglicht ein einfaches, kostengünstiges Hosting auf beliebigen statischen Webservern.

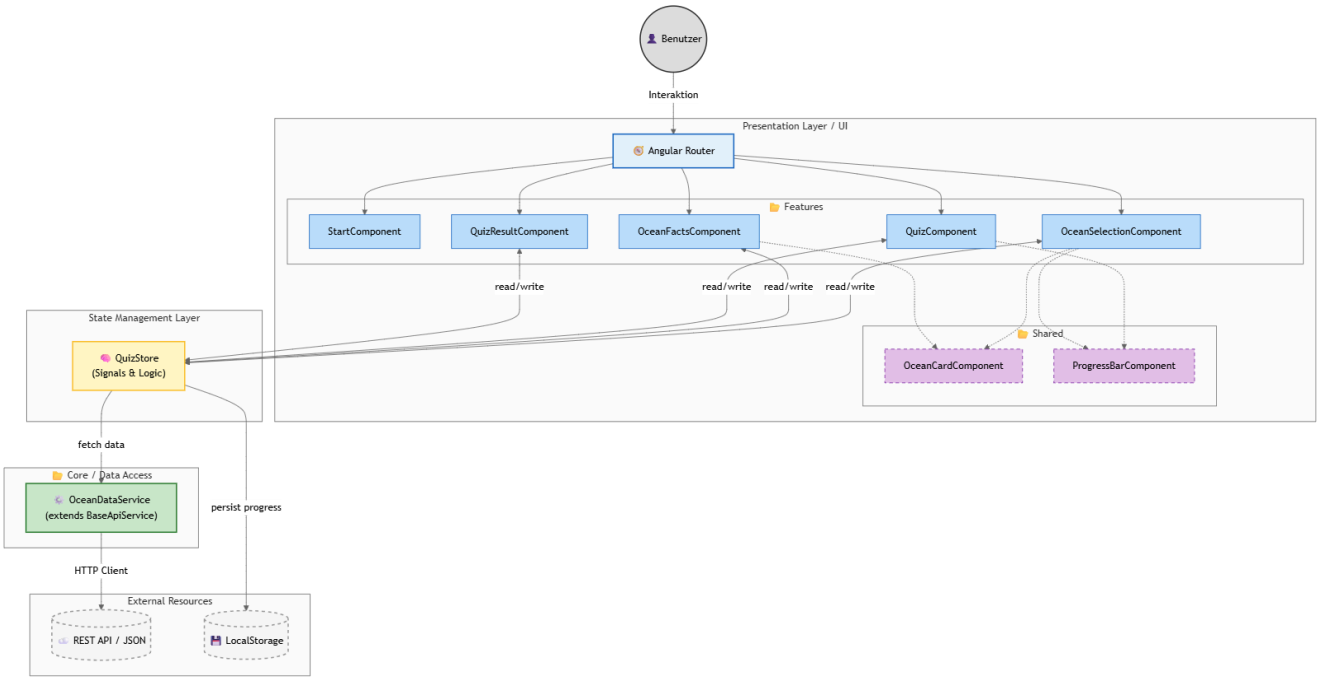


Abb.3.2 Die Architektur orientiert sich am Model-View-ViewModel (MVVM) Muster, welches durch die moderne Angular-Architektur (Signals) effizient umgesetzt wird.

Technologie-Entscheidungen:

Angular 21: In Abstimmung mit dem Ausbilder wurde Version 21 gewählt, um das Projekt „State of the Art“ umzusetzen[2] und von neuesten Optimierungen und der Erfahrung zu profitieren.

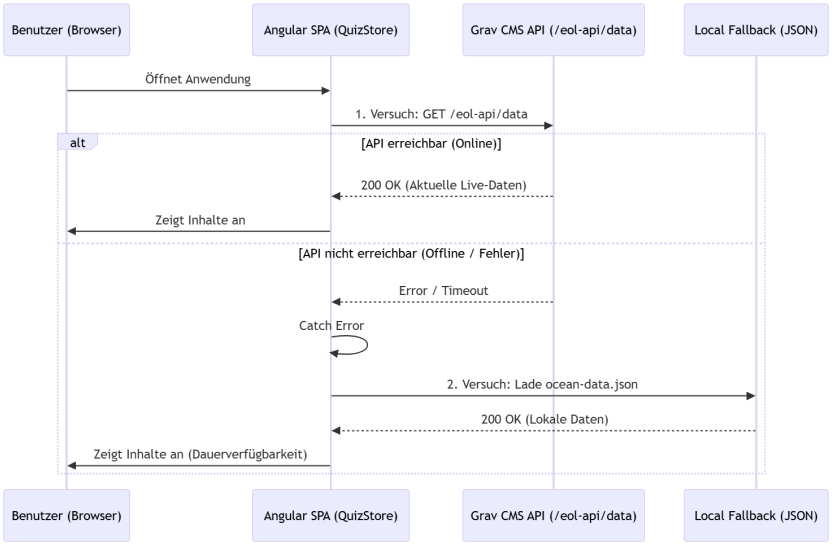
Client-Side Rendering (CSR): Die Entscheidung für CSR sorgt für ein flüssiges, App-ähnliches Nutzererlebnis ("App-Feel"). Nach dem initialen Laden reagiert die Anwendung sofort auf Benutzereingaben, ohne Wartezeiten durch Server-Roundtrips.

State Management (QuizStore): Dieser nutzt die neuen Angular Signals für reaktive Programmierung. Dies ermöglicht eine sehr feingranulare Aktualisierung der Benutzeroberfläche.

Datenhaltung (Hybrid: API-First mit Fallback): Die Architektur folgt einem robusten "API-First"-Ansatz.

- **Primär (Online/Connected):** Die Anwendung versucht initial, die aktuellen Lehrinhalte dynamisch über die REST-Schnittstelle (`/eol-api/data`) des eigens entwickelten Grav-Plugins (`eol-manager`) zu laden. Dies ermöglicht Lehrkräften, Inhalte (z.B. Quizfragen) in Echtzeit zu aktualisieren, ohne die App neu deployen zu müssen.

- **Sekundär (Offline/Fallback):** Ist die API nicht erreichbar (z.B. Offline-Betrieb auf Tablets im Klassenzimmer oder fehlendes Backend), schaltet die Applikation nahtlos und für den Nutzer unsichtbar auf die lokal gebündelte `ocean-data.json` um. Dies garantiert eine 100%ige Verfügbarkeit.



[2] [1] Angular Dokumentation

Tailwind CSS & BEM: Die Gestaltung erfolgt durch eine Kombination aus Tailwind CSS[3] für konsistente Design-Tokens (Farben, Abstände) und der BEM-Methodik (Block-Element-Modifier)[4] für die Struktur. Diese Strategie verhindert überladenen HTML-Code ("Class Soup") und ermöglicht eine saubere Kapselung (Encapsulation) der Styles innerhalb der Angular-Komponenten.

Erweiterung der Architektur (Refactoring): Um die Wartbarkeit und Erweiterbarkeit zu maximieren, wurde die Architektur im Realisierungsverlauf in Core-, Shared- und Feature-Module unterteilt.

3.3 UI/UX Design

Das Design wurde speziell für Kinder entwickelt:

- **Farbpalette:** Helle, freundliche Farben (Blau-, Türkis- und Sandtöne im Thema Ozean.
- **Typografie:** Gute Lesbarkeit durch serifenlose Schriftarten.
- **Navigation:** Große Knöpfe, intuitive "Weiter"-Buttons, visuelles Feedback
- **Barrierefreiheit (Accessibility)**: WCAG-Standards (z.B. Farbkontraste, Tastaturnavigation/Tab-Index) zur Gewährleistung der Barrierefreiheit.



Abb.3.3 Ein Mockup der Anwendungsbildschirme

[3] [3] Tailwind Dokumentation
[4] [8] BEM- Methodik

3.4 Datenmodell

Die Daten werden in einer JSON-Struktur gehalten, um Flexibilität zu gewährleisten und eine Datenbank-Abhängigkeit für diesen Prototypen zu vermeiden.

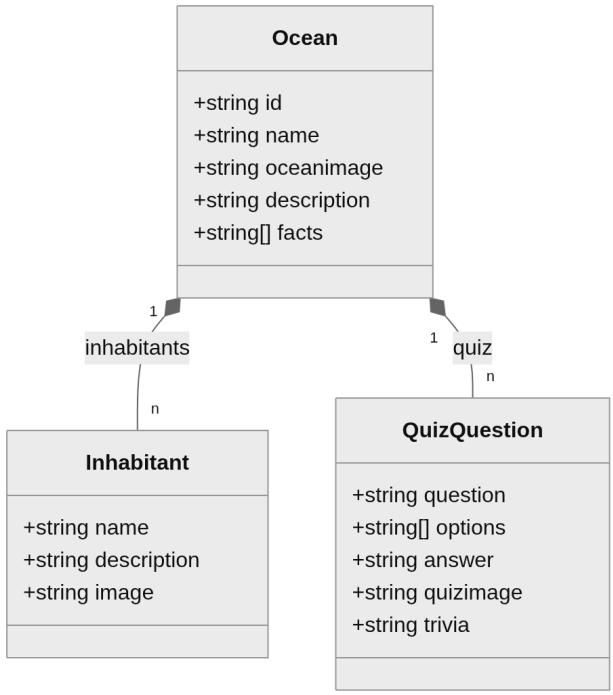


Abb.3.4 Da eine dokumentenorientierte Speicherung (JSON) statt einer relationalen Datenbank genutzt wird, zeigt Abb. 3.4 die hierarchische JSON-Objektstruktur und keine relationalen Tabellenverknüpfungen.

```
{
  "id": "pacific",
  "name": "Pazifischer Ozean",
  "facts": [ ... ],
  "inhabitants": [ ... ],
  "quiz": [
    {
      "question": "Wie tief ist der Marianengraben?",
      "options": ["11.000m", "5.000m", "2.000m", "8.000m"],
      "answer": "11.000m",
      "trivia": "Der tiefste Punkt der Erde."
    }
  ]
}
```

Listing 3.4. Beispiel datenblock aus der ocean-data.json

3.5 Klassendiagramm (UML)

Das Klassendiagramm verdeutlicht die Abhängigkeiten zwischen den Standalone Components, dem QuizStore und den Daten-Services.

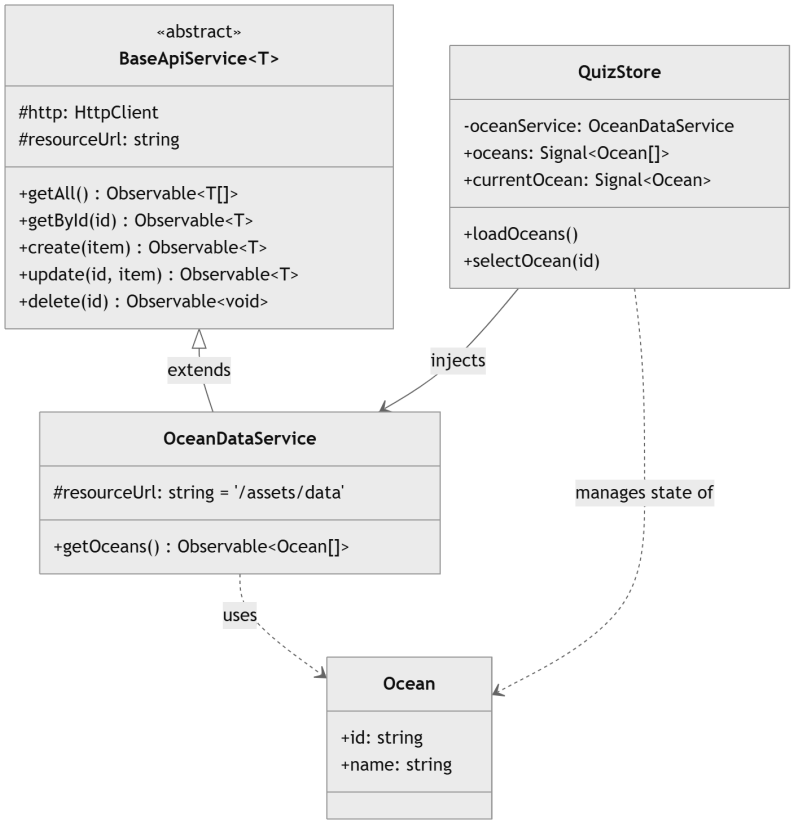


Abb.3.5 Das Klassendiagramm zeigt die Architektur der Anwendung, wobei der QuizStore als zentraler State-Manager ("Service with Signals") fungiert, auf den die Komponenten zugreifen.

3.6 Datenschutz & Sicherheit (Privacy by Design)

Da sich die Anwendung an Kinder richtet, hat der Datenschutz höchste Priorität.

- Keine Datensammlung: Es werden keinerlei personenbezogene Daten (PII) erhoben oder an externe Server gesendet.
- Lokale Speicherung: Der Lernfortschritt wird ausschließlich im LocalStorage[5] des Browsers auf dem Endgerät des Nutzers gespeichert.
- Offline-Fähigkeit: Die Anwendung lädt keine externen Tracker oder Analyse-Tools nach.
- Keine Cookies. Daten werden nur lokal via LocalStorage gespeichert.

[5] [4] Web Storage API (LocalStorage)

4. Realisierung

4.1 Entwicklungsumgebung

Betriebssysteme: Windows Server 2025 und Rocky Linux 10

Laufzeitumgebung & Paketmanagement:

- Node.js (verwaltet via NVM - Node Version Manager).
- NPM (Node Package Manager).

Framework & Core-Technologien: Angular CLI (Version 21.0.3).

Entwicklungswerkzeuge (IDEs):

- Visual Studio Code (Erweiterung: Angular Language Service).
- Sublime Text 4 (Editor für schnelle Anpassungen).

Projektmanagement & Versionierung:

- **GitHub Repository:** Zur Quellcodeverwaltung und Versionierung.
- **GitHub Projects:** Nutzung als digitales **Kanban-Board** zur Aufgabensteuerung und Issue-Tracking (Verknüpfung von Commits mit Tickets).

Testing & Debugging: Google Chrome (DevTools), Google Lighthouse

Doku & Assets: Mermaid (Diagramm-Erstellung), WikiCommons (Bild-Platzhalter).

4.2 Implementierung der Hauptkomponenten

4.2.1 Standalone Components

Das Projekt setzt vollständig auf Standalone Components, um die Komplexität von NgModules zu vermeiden.

```
@Component({
  selector: 'app-ocean-facts',
  standalone: true,
  imports: [ImageFallbackDirective],
  styleUrl: './ocean-facts.component.css',
  template: `
 ... })
export class OceanFactsComponent { ... }
```

Listing 4.2.1 Die OceanFactsComponent mit Inline-Template, die Abhängigkeiten direkt im Component-Decorator definiert.

4.2.2 State Management via QuizStore

Für die Verwaltung des Anwendungszustands wurde das **„Service with Signals“**[6]-Pattern verwendet. Dabei handelt es sich um einen global verfügbaren Angular Injectable Service, der den Zustand in privaten WritableSignals hält. Nach außen werden diese Daten ausschließlich als schreibgeschützte Signale (ReadonlySignal) sowie als abgeleitete Werte (computed) exponiert. Dies ermöglicht eine strikte Kapselung und reaktive Datenhaltung mit reinen Angular-Bordmitteln, ohne die Komplexität externer State-Management-Bibliotheken einzuführen.

```
@Injectable({
  providedIn: 'root'
})
export class QuizStore {
  private readonly oceanDataService = inject(OceanDataService);
  private readonly STORAGE_KEY = 'eol_progress';

  // --- State ---
  private readonly _oceans = signal<Ocean[]>([]);
  private readonly _currentOceanId = signal<string | null>(null);
...
  // --- Public Readonly Signals ---
  readonly oceans: Signal<Ocean[]> = this._oceans.asReadonly();
  readonly currentOceanId: Signal<string | null> =
this._currentOceanId.asReadonly();
...
  readonly currentOcean = computed(() =>
    this._oceans().find(o => o.id === this._currentOceanId())
  );
```

Listing 4.2.2: QuizStore: Streng getrennter interner State, API und Computed Signals.

[6] [2] *Angular Signals Guide*

4.2.3 Routing & Navigation

Das Routing ist in der Konfigurationsdatei `app.routes.ts` definiert. Es steuert die Navigation zwischen den Hauptansichten Startseite, Ozean-Auswahl, Fakten und Quiz. Dynamische Parameter, wie die ID des gewählten Ozeans, werden direkt über die URL definiert (`path: 'facts/:id'`) und an die Zielkomponente übergeben. Zusätzlich sorgt eine Wildcard-Route (`**`) dafür, dass ungültige URLs abgefangen und automatisch auf die Startseite umgeleitet werden.

```
export const routes: Routes = [
    { path: '', component: StartComponent },
    { path: 'selection', component: OceanSelectionComponent },
    { path: 'facts/:id', component: OceanFactsComponent },
    { path: 'quiz', component: QuizComponent },
    { path: 'result', component: QuizResultComponent },
    { path: '**', redirectTo: '' }
];
```

Listing 4.2.3 Definition der Anwendungsrouten inklusive Parameterübergabe und Fallback-Umleitung.

4.2.4 CSS-Architektur und Styling-Strategie

Um die Wartbarkeit und Lesbarkeit des Codes zu maximieren, wurde im Projektverlauf ein Refactoring der CSS-Architektur durchgeführt.[7] Anstatt eines reinen Utility-First-Ansatzes mit überladenen HTML-Templates wurde eine semantische Strategie gewählt:

- **BEM-Namenskonvention:** CSS-Klassen folgen strikt dem Schema `block__element--modifier` (z. B. `.ocean-card` oder `.quiz-btn--correct`). Dies macht sofort ersichtlich, welches Styling zu welcher Komponente gehört.[8]
- **Separation of Concerns:** Das HTML-Template bleibt schlank und lesbar. Die visuellen Eigenschaften werden in die zugehörige CSS-Datei ausgelagert.
- **Tailwind @apply & Hybrid-Styling:** Tailwind-Utilities werden in CSS-Klassen mittels `@apply` kombiniert. Komplexe Effekte (z.B. Glassmorphismus) nutzen zusätzlich native CSS-Regeln/Variablen. Dies verbindet Design-System-Vorteile mit klassischer Flexibilität.
- **Component Encapsulation:** Durch Angulars View Encapsulation bleiben Styles lokal auf die Komponente beschränkt, was globale Seiteneffekte verhindert.
- **Tailwind CSS v4 & Native CSS Variablen:** Tailwind CSS v4 ersetzt die JavaScript-Konfiguration (tailwind.config.js) durch natives CSS. Design-Tokens werden nun performant mittels der neuen **@utility Direktive** direkt im CSS definiert, was eine nahtlose Integration von BEM-Klassen und Tailwind-Utilities ermöglicht.
- *Beispiel:* Die Klasse `.glass-card` wird als Custom Utility definiert, die sowohl CSS-Filter (Blur) als auch Tailwind-Klassen (`@apply rounded-2xl`) kombiniert. Dies reduziert die HTML-Komplexität drastisch und nutzt moderne Browser-Features optimal.

Beispiel:

Vorher (Utility-First im HTML):

```
// ocean-facts.component.ts
<div class="p-6 rounded-2xl bg-white/25 border border-white/30 shadow-lg
hover:scale-105">
```

[7] Refactoring bezeichnet die Strukturverbesserung von Quellcode unter Beibehaltung des verhaltenen Programmverhaltens.
[8] [8] BEM- Methodik

Nachher (BEM + @apply):

```
// ocean-facts.component.ts
styleUrl: './ocean-facts.component.css',
...
<div class="ocean-card"> ... </div>
```

```
// ocean-facts.component.css
@reference "../../../styles.css";
```

```
// ocean-card.component.css
@reference "../../../../styles.css";
/* Block: Ocean Card */
.ocean-card {
    /* Lokale Nutzung der globalen Variablen ist okay für Konsistenz */
    background: var(--glass-bg, rgba(255, 255, 255, 0.25));
    backdrop-filter: blur(12px);
    -webkit-backdrop-filter: blur(12px);
    border: 1px solid var(--glass-border, rgba(255, 255, 255, 0.3));
    box-shadow: var(--glass-shadow, 0 8px 32px 0 rgba(31, 38, 135, 0.37));
    @apply rounded-2xl p-6 cursor-pointer transform transition-all
duration-300 relative w-full sm:w-80 md:w-96;
    @apply hover:scale-105 hover:bg-white/40;
    @apply animate-pop-in;
```

Listing 4.2.4: Refactoring zu semantischem CSS: Kombination aus nativen CSS-Variablen für das Design und Tailwind @apply für Layout und Interaktion.

4.3 Entwicklung des Backend-Moduls

Um die Anforderung der "einfachen Integration in CMS-Systeme" (vgl. 1.2) technisch vollumfänglich zu lösen, wurde ein maßgeschneidertes Plugin (`eol-manager`) für das Flat-File-CMS **Grav** entwickelt.

Funktionsumfang des Plugins:

- **REST-API:** Bereitstellung der Endpunkte `GET /eol-api/data` für die Angular-App und `POST /eol-api/save` zur Datenpflege.
- **Dashboard-UI:** Eine grafische Oberfläche (entwickelt mit Twig & Vanilla JS), die es Lehrkräften ermöglicht, Ozean-Fakten und Quizfragen über Formulare zu bearbeiten, ohne JSON-Dateien manuell editieren zu müssen.
- **Failsafe-Handling:** Implementierung einer konfigurierbaren Failsafe Strategie, damit die Angular-SPA (vom Client) sicher auf die API zugreifen darf.

```
    protected function handleApi($route)
    {
        $dataService = new DataService($this->grav);
        $method = $_SERVER['REQUEST_METHOD'];
        $config = $this->grav['config']->get('plugins.eol-manager');

        // CORS Handling
        if (!empty($config['cors']['enabled'])) {
            $origin = $config['cors']['allow_origin'] ?? '*';
            $methods = $config['cors']['allow_methods'] ?? 'GET, POST,
OPTIONS';
            $headers = $config['cors']['allow_headers'] ?? 'Content-Type,
Authorization';

            header("Access-Control-Allow-Origin: " . $origin);
            header("Access-Control-Allow-Methods: " . $methods);
            header("Access-Control-Allow-Headers: " . $headers);
        }
```

Listing 4.3: eol-manager.php zeigt Das Failsafe handling mittel CORS

4.4 Herausforderungen & Lösungen

Während der Realisierungsphase traten verschiedene technische und konzeptionelle Herausforderungen auf, die durch spezifische Lösungsansätze bewältigt wurden. Die folgende Tabelle fasst diese zusammen.

Herausforderung	Lösungsansatz	Ergebnis
Persistierung des Fortschritts Datenverlust beim Neuladen der Seite (F5).	Nutzung eines Angular `effect` innerhalb des `QuizStore`. Dieser überwacht das `completedOceans`-Signal und synchronisiert Änderungen automatisch mit dem `localStorage`. Beim App-Start wird der Speicher ausgelesen.	Lernfortschritt (Sterne) bleibt dauerhaft erhalten, auch offline.
Code-Qualität / Linting Inkompatibilität von ESLint mit Angular 21 (RC).	Verzicht auf externe Linter. Stattdessen konsequente Nutzung der strikten Angular-Compiler-Checks (`strict: true`) in der `tsconfig.json` sowie durchgeführte manuelle Code-Reviews.	Hohe Code-Qualität durch Disziplin, keine Build-Fehler.
Signals Einarbeitung Paradigmenwechsel von klassischen Services zu Signals.	Intensive Nutzung der offiziellen Dokumentation. Umstellung auf deklarativen Code mittels `computed()` für abgeleitete Zustände anstelle von imperativen manuellen Subscriptions.	Sehr performanter, reaktiver Code mit deutlich weniger Boilerplate.
Kindgerechte UX Interface muss ohne viel Text verständlich sein.	Nutzung von großen Icons, intuitiven Farben (Grün/Rot) und Verzicht auf komplexe Menüstrukturen.	Positive Rückmeldung bei ersten Tests mit der Zielgruppe (intuitiv bedienbar).
Datenmodellierung Abbildung komplexer Relationen in Flat-File JSON.	Design einer verschachtelten JSON-Struktur mit Arrays für Fakten/Bewohner, die zur Laufzeit typisiert eingelesen wird.	Flexibles Datenmodell ohne Backend-Zwang, leicht erweiterbar.

Tab.4.3 Übersicht technischer und konzeptioneller Herausforderungen sowie deren Lösungen.

5. Qualitätssicherung

Teststrategie: Herausforderung "Bleeding Edge"

Das Projekt setzte auf die neueste Technologie, was die Qualitätssicherung anfangs durch Inkompatibilitäten etablierter Testing-Frameworks (wie Karma, Jest) mit dem neuen Build-System erschwerte.

Glücklicherweise erhielten die modernen Testing-Tools Vitest und Playwright kurz vor der Realisierungsphase entscheidende Updates, die eine Integration ermöglichten. Dadurch konnte die Teststrategie von rein manuellen Tests auf eine vollautomatisierte Hybrid-Strategie umgestellt werden.

Unit-Testing mit Vitest

Anstelle des veralteten Karma-Runners wurde **Vitest** integriert. Vitest bietet eine native Unterstützung für ESM (ECMAScript Modules) und ist signifikant schneller als traditionelle Runner.

- **Testabdeckung:** Es wurden Unit-Tests für die kritische Business-Logik (Store) und die Komponenten-Interaktionen erstellt.
- **Signals Testing:** Da Angular Signals synchron arbeiten, konnte auf komplexe `fakeAsync`-Wrapper oft verzichtet werden.

```
EhlenAngularProjekt3  npm test
> eol@0.0.0 test
> vitest
 DEV  v4.0.15 D:/DEV/EOL/EhlenAngularProjekt3/EhlenAngularProjekt3
 ✓ src/app/services/ocean.data.service.spec.ts (3 tests) 26ms
 ✓ src/app/store/quiz.store.spec.ts (7 tests) 34ms
 ✓ src/app/shared/components/ocean-card/ocean-card.component.spec.ts (7 tests) 114ms
 ✓ src/app/features/quiz/quiz.component.spec.ts (6 tests) 111ms
 Test Files  4 passed (4)
      Tests  23 passed (23)
   Start at  17:01:52
   Duration  3.15s (transform 2.59s, setup 4.84s, import 455ms, tests 285ms,
environment 5.50s)
 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

Listing. 5.1 Erfolgreiche Unit-Tests im Terminal mit Vitest

End-to-End Testing mit Playwright

Für die Simulation echter Nutzerinteraktionen wurde Playwright implementiert. Dies ermöglichte das Testen der gesamten "User Journey" von der Startseite bis zum Quiz-Abschluss in echten Browser-Umgebungen (Chromium, WebKit).

Testdurchführung & Ergebnisse

Die Tests wurden sowohl lokal als auch in der CI/CD-Pipeline ausgeführt.

Typ	Komponente / Flow	Beschreibung	Status
Unit	QuizStore	Initialisierung, Laden der Daten, State-Updates.	Pass
Unit	QuizComponent	Rendern von Fragen, Feedback-Anzeige (Richtig/Falsch).	Pass
E2E	Quiz Flow	Kompletter Durchlauf: Start -> Ozeanwahl -> Quiz -> Ergebnis.	Pass
E2E	Smoke Test	Prüfung, ob die App grundsätzlich lädt (Smoke Test).	Pass

Tab 5.2 Playwright HTML-Reports

5.1 Automatisierte Qualitätssicherung mittels CI/CD

Die `package.json` wurde um entsprechende Skripte erweitert, um die Qualitätssicherung zu standardisieren:

- `npm test`: Führt die Unit-Tests mit Vitest aus.
- `npm run test:e2e`: Startet die Playwright E2E-Tests.
- `npm run lint`: Statische Codeanalyse (ESLint wurde dank Updates auf Angular 21 angepasst).

5.2 Google Lighthouse Audit

Zur objektiven Bewertung der technischen Qualität und der Benutzerfreundlichkeit wurde ein automatisierter Audit mittels **Google Lighthouse** (Chrome DevTools)[9] durchgeführt. Besonderer Fokus lag dabei auf der **Performance** auf mobilen Endgeräten (Tablets) und der Barrierefreiheit (**Accessibility**), da die Zielgruppe Kinder eine einfache Bedienung erfordert[10].

Ergebnisse: Die Anwendung erzielt durch die moderne Architektur (Angular 21 + Tailwind CSS) überzeugende Werte:

- **Performance (85/100):** Durch die Nutzung von Angular Signals ist die Render-Performance exzellent. Einzig die Verwendung von Bildern im PNG-Format (statt WebP) verhindert hier eine Bestnote.
- **Accessibility (100/100):** Alle interaktiven Elemente sind per Tastatur (Tab-Taste) erreichbar. Bilder besitzen sinnvolle Alt-Texte.
- **Best Practices (100/100):** Die Anwendung nutzt modernes HTTPS (lokal simuliert) und verzichtet auf veraltete Web-APIs.

Fazit des Audits: Der Lighthouse-Report bestätigt, dass die technische Entscheidung für eine schlanke SPA ohne schwere Drittanbieter-Bibliotheken (außer Tailwind/Confetti) korrekt war. Die App ist auch auf schwächerer Schul-Hardware flüssig nutzbar.

[9] [5] *Lighthouse Overview*
[10] [7] *WCAG 2.2 Overview*

6. Wirtschaftlichkeitsbetrachtung

6.1 Soll-Ist-Vergleich (Zeit)

*Die 71 IHK-Stunden bildeten den Rahmen; das angestrebte technische Niveau machte jedoch einen wesentlich höheren privaten Lern- und Entwicklungsaufwand nötig. Die **tatsächliche Gesamtzeit** ist unter **A.9 Realer Zeitaufwand** im Anhang detailliert.*

Die Projektdurchführung verlief weitgehend nach Plan. Während die frühen Phasen (Analyse & Entwurf) zeiteffizienter als veranschlagt abgeschlossen werden konnten, erforderte die technische Umsetzung aufgrund der neuen Angular-Features einen erhöhten Aufwand.

Phase	Geplant (h)	Tatsächlich (h)	Abweichung
Analyse & Planung	11	8	-3 h
Entwurf	12	8	-4 h
Implementierung	28	35	+7 h
Qualitätssicherung	9	11	+2 h
Dokumentation	10	9	+1 h
Gesamt	**70 h**	**71 h**	**+1 h**

Tab.6.1: Gegenüberstellung der geplanten und tatsächlich aufgewendeten Projektstunden..

Begründung der Abweichungen:

- **Entwurf (-4 h):** Durch die klare Anforderungsdefinition und schnelle Abstimmungswege beim UI-Design konnte in dieser Phase Zeit eingespart werden.
- **Implementierung (+7 h):** Die Einarbeitung in die neuen Angular Signals sowie das Refactoring der CSS-Architektur waren zeitintensiver als ursprünglich kalkuliert.
- **Qualitätssicherung (+2 h):** Obwohl die Umstellung auf Vitest und Playwright automatische Tests ermöglichte (siehe 5.1), benötigten deren initiale Einrichtung und Konfiguration mehr Zeit als für reine Standard-Tests geplant.

6.2 Nachkalkulation (Kosten)

Da die geplante Zeitvorgabe von 70 Stunden mit 71 Stunden (+1,4 %) nahezu exakt eingehalten wurde, entsprechen die tatsächlichen Kosten weitestgehend den Plankosten.

- **Personalkosten (Ist):** 71 h × 9,00 €/h = **639,00 €**
- **Sachmittel (Ist):** pauschal **150,00 €**
- **Gesamtkosten (Ist): 789,00 €**

6.3 Amortisationsrechnung

Die Wirtschaftlichkeit des Projekts wird durch einen Vergleich der neuen digitalen Lösung mit dem bisherigen analogen Prozess der NGO (Druck von Lern-Broschüren) ermittelt.

Abgrenzung der Kosten (Hardware): In dieser Betrachtung werden keine Investitionskosten für Hardware (Tablets/PCs) angesetzt.

- Begründung: Die Zielgruppe (Grundschulen) wurde im Rahmen des "Digitalpakt Schule" bereits weitestgehend mit Tablets ausgestattet.
- Sicht der NGO: Für den Auftraggeber fallen somit keine Hardwarekosten an. Die App nutzt die vorhandene Infrastruktur der Schulen ("Bring Your Own Device" / Schulausstattung).

Ausgangslage (Ist-Zustand „Analog“): Die NGO druckt jährlich ca. 2.000 Lernbroschüren.

- Druckkosten & Logistik: 2.000,00 € / Jahr

Zielzustand (Soll-Zustand „App“): Die Web-App ersetzt die physischen Broschüren vollständig.

- Jährliche Betriebskosten (Hosting & Wartung): 260,00 €

Berechnung der Amortisationsdauer:

$$Jährliche\ Einsparung\ =\ 2.000,00\ €\ -\ 260,00\ €\ =\ 1.740,00\ €$$

$$Amortisationsdauer\ =\ \frac{Einmalkosten\ Projekt}{Jährliche\ Einsparungen} \approx\ 0,45\ Jahre$$

Fazit: Da das Projekt im Rahmen einer Umschulung mit niedrigen internen Verrechnungssätzen realisiert wurde, amortisiert sich die Entwicklung bereits nach **weniger als 6 Monaten**. Ab dem zweiten Halbjahr erzielt der Kunde bereits reine Einsparungen. Selbst bei Ansatz eines marktüblichen Junior-Entwickler-Stundensatzes (ca. 35–45 €) wäre das Projekt innerhalb von 1,5 Jahren amortisiert, was die hohe Wirtschaftlichkeit der „Make“-Entscheidung unterstreicht.

Qualitative Bewertung:
Neben der monetären Einsparung bietet die App eine deutlich höhere Reichweite und Interaktivität. Sollten durch die moderne Außenwirkung der App auch nur zwei zusätzliche Schulprojekte pro Jahr gewonnen werden (Deckungsbeitrag je ca. 500 €), hätte sich die Investition bereits im ersten Jahr vollständig amortisiert.

7. Fazit & Ausblick

7.1 Zusammenfassung

Das Projekt „Earth Ocean Learning“ konnte erfolgreich und innerhalb des geplanten Zeitrahmens realisiert werden. Sämtliche funktionalen Anforderungen – vom interaktiven Lernmodus über die Quiz-Logik bis hin zur persistenten Fortschrittsspeicherung – wurden vollständig erfüllt. Das Ergebnis ist eine stabile, performante Web-Applikation, die durch ihr responsives Design eine durchgängig positive User Experience (UX) auf unterschiedlichen Endgeräten gewährleistet.

Besonders hervorzuheben ist die erfolgreiche Umsetzung der Hybrid-Architektur. Durch das zusätzlich entwickelte Backend-Modul (Grav Plugin) konnte ein voll funktionsfähiges Lehrer-Dashboard bereitgestellt werden, welches die Verwaltung der Lerninhalte deutlich vereinfacht und die ursprünglichen Anforderungen übertrifft.

7.2 Lessons Learned

Die Durchführung dieses Projekts an der technologischen Speerspitze ("Bleeding Edge") lieferte weit mehr als nur fachliche Übung. Sie erforderte eine kontinuierliche Bewertung von Architekturentscheidungen unter den Aspekten Wartbarkeit, Performance und Stabilität. Im Folgenden werden die zentralen Erkenntnisse detailliert reflektiert.

Professioneller Entwicklungsworkflow (Git & GitHub)

- **Branching-Strategie:** Die konsequente Nutzung von Feature-Branches (z. B. `feature/quiz-logic`, `style/ocean-cards`) schützte den `main`-Branch effektiv vor instabilem Code. Dies simulierte bereits im Einzelprojekt erfolgreich professionelle Team-Workflows.
- **Conventional Commits:** Die Einhaltung standardisierter Commit-Messages (z. B. `feat: add timer logic`, `fix: layout overlap`) generierte eine automatisch nachvollziehbare Historie. Dies erwies sich bei der späteren Fehlersuche als essenziell.
- **Atomare Commits:** Kleine, logisch abgeschlossene Commits (statt großer Änderungen) erleichterten Debugging und Reverts erheblich, da Fehler einfacher zu isolieren waren.

Projektmanagement & Strategie

- **MVP-Fokus (Qualität vor Quantität):** Durch die strikte Fokussierung auf den Kern (Anzeigen, Quizzen, Speichern) blieben zum Projektende genügend Ressourcen für die Qualitätssicherung (z. B. Lighthouse-Optimierung auf 100 % Accessibility). Ein stabiles, sauber implementiertes MVP ist wertvoller als eine funktionsüberladene, aber fehleranfällige Anwendung.
- **Agilität & Tooling: Vom Hindernis zum Upgrade** Eine der wichtigsten Lektionen war der Umgang mit der anfänglichen Inkompatibilität etablierter Testing-Tools (Karma, Jasmine) mit Angular 21. Anstatt bei der (aus der Not geborenen) manuellen Teststrategie zu verharren, beobachtete ich die Entwicklung des Ökosystems aktiv. Die Entscheidung, kurz vor Projektende auf die gerade kompatibel gewordenen Tools **Vitest** und **Playwright** umzusteigen, erwies sich als entscheidender Qualitätsgewinn.
    - **Erkenntnis:** Moderne Softwareentwicklung erfordert Flexibilität. Das Festhalten an veralteten Standards hätte zu einer schlechteren Wartbarkeit geführt. Der Mut zum Technologiewechsel ermöglichte stattdessen eine hochmoderne, vollautomatisierte Testumgebung, die signifikant performanter ist als der ursprüngliche Standard.
- **Dokumentation & Journaling:** Das zeitgleiche Führen eines Projekttagebuchs machte komplexe Entscheidungen (z. B. „Warum JSON statt SQL?“) im Nachhinein transparent. Eine erst am Ende erstellte Dokumentation hätte die wichtigen Lösungswege und überwundenen Hürden der Anfangsphase vernachlässigt.

Technische Erkenntnisse & Refactoring

- Der anfängliche „Utility-First“-Ansatz führte zu unübersichtlichem Code. Ein Refactoring hin zur BEM-Methodik und der Tailwind `@apply`-Direktive löste dieses Problem. Styles werden nun in semantischen Klassen gekapselt, was die Trennung von Struktur und Design ermöglicht, Design-Änderungen zentralisiert und die Wartung vereinfacht.
- **Trennung von Zuständigkeiten (Store vs. Service):** Die anfängliche Vermischung von Datenbeschaffung und State-Management wurde durch eine strikte Trennung in den zustandslosen OceanDataService (reiner HTTP-Abruf) und den zustandsbehafteten QuizStore (hält Signals) behoben.[11] Diese Orientierung an Enterprise-Mustern („State vs. Data“) erhöhte die Testbarkeit und eliminierte Seiteneffekte durch klare Verantwortlichkeiten.
- **Angular Signals:** vereinfachten die Anwendungslogik und machten sie robuster (keine manuellen Subscriptions mehr). Mittels `computed()` und `effect()` nutzen sie die OnPush-Change-Detection effizienter, vermeiden unnötiges Rendern ("Zone Pollution") und steigern die Performance, besonders mobil.
- **Performance-Optimierung (Assets & LCP):** Lange Ladezeiten hochauflösender Ozean-Bilder wurden durch Konvertierung nach WebP und Priorisierung kritischer Ressourcen mittels `fetchpriority="high"`[12] proaktiv behoben, um schnelle Ladezeiten bei hoher visueller Qualität zu gewährleisten.

[11] [2] *Angular Signals Guide*
[12] [1] *Angular Documentation*

7.3 Ausblick

Für zukünftige Versionen sind folgende Erweiterungen in der Roadmap:

- **Backend-Integration & Benutzerkonten (Classroom-Management)** Die aktuelle Speicherung im LocalStorage ist gerätegebunden. Die Anbindung eines dedizierten Backends (z. B. via NestJS oder Firebase) würde persistente Benutzerkonten ermöglichen. Dies ist die Grundlage für ein „Lehrer-Dashboard“, über das Lehrkräfte den Lernfortschritt ihrer Klasse einsehen und gezielt Hausaufgaben zuweisen können.
- **Barrierefreiheit & Text-to-Speech (Inklusion)** Da die Zielgruppe auch Kinder umfasst, deren Lesekompetenz noch nicht vollständig ausgeprägt ist, soll eine Vorlesefunktion (Text-to-Speech) integriert werden. Dies erhöht die Barrierefreiheit und ermöglicht auditives Lernen.
- **Internationalisierung (i18n)** Vorbereitung der Architektur für Mehrsprachigkeit (Angular i18n), um die Anwendung auch im englischsprachigen Raum oder im Fremdsprachenunterricht einsetzen zu können.

7.4 Projektabnahme und Übergabe

Zum Abschluss des Projekts erfolgte am 09.12.2025 die offizielle Abnahme der Software durch den Auftraggeber, vertreten durch Herrn Dr. Uwe Umwelt. Grundlage für die Abnahme bildete das im Vorfeld definierte Testprotokoll sowie der Soll-Ist-Vergleich der Anforderungen aus dem FCPP. In einer gemeinsamen Sitzung wurden die Kernfunktionen – insbesondere die Quiz-Logik, die Persistierung des Lernfortschritts im LocalStorage sowie das responsive Verhalten auf Endgeräten – erfolgreich demonstriert.

Alle im Abnahmeprotokoll (siehe Anhang) aufgeführten Testfälle (TF01 bis TF06) wurden erfolgreich validiert. Der Auftraggeber bestätigte die Mängelfreiheit der Version 1.0 und erteilte die Freigabe für den produktiven Einsatz.

Die Übergabe des Quellcodes und des Benutzerhandbuchs erfolgte digital.

A. Anhang

A.0 Quellen- und Literaturverzeichnis

- **[1] Angular Team:** *Angular Documentation (v21)*. URL: https://angular.dev (Zuletzt geprüft: 08.12.2025). [Verwendet für Core-Framework, Signals und Routing].
- **[2] Angular Team:** *Angular Signals Guide*. URL: https://angular.dev/guide/signals (Zuletzt geprüft: 08.12.2025). [Grundlage für das State Management im QuizStore].
- **[3] Tailwind Labs:** *Tailwind CSS v4 Documentation*. URL: https://tailwindcss.com/docs (Zuletzt geprüft: 08.12.2025). [Verwendet für Styling und die neue `@theme`-Konfiguration].
- **[4] Mozilla Developer Network (MDN):** *Web Storage API (LocalStorage)*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage (Zuletzt geprüft: 05.12.2025). [Verwendet für die Persistierung des Fortschritts].
- **[5] Google Developers:** *Lighthouse Overview*. URL: https://developer.chrome.com/docs/lighthouse/ (Zuletzt geprüft: 07.12.2025). [Verwendet für Qualitäts-Audit und Accessibility-Check].
- **[6] Catdad (NPM):** *canvas-confetti Documentation*. URL: https://www.npmjs.com/package/canvas-confetti (Zuletzt geprüft: 06.12.2025). [Bibliothek für visuelle Effekte im Quiz-Ergebnis].
- **[7] W3C Web Accessibility Initiative (WAI):** *WCAG 2.2 Overview*. URL: https://www.w3.org/WAI/standards-guidelines/wcag/ (Zuletzt geprüft: 02.12.2025). [Grundlage für Barrierefreiheits-Anforderungen].
- **[8] GetBEM:** *BEM — Block Element Modifier*. URL: https://getbem.com/ (Zuletzt geprüft: 01.12.2025). [Methodik für die CSS-Architektur].
- **[9] Mermaid.JS:** *Mermaid lets you create diagrams and visualizations*. URL: https://mermaid.js.org/intro/ (Zuletzt geprüft: 06.12.2025). [Tool zum Erstellen von komplexen Diagrammen mittels Markup-Language].
- **[10] Vitest Dev Team:** *Vitest Documentation*. URL: https://vitest.dev (Zuletzt geprüft: 09.12.2025). [Verwendet für Unit-Tests als moderner Ersatz für Karma].
- **[11] Microsoft:** *Playwright Documentation*. URL: https://playwright.dev (Zuletzt geprüft: 09.12.2025). [Verwendet für automatisierte End-to-End (E2E) Tests].

A.0.1 Bildquellen

- **WikiCommons / Public Domain:** Platzhalter-Bilder für Ozeane und Icons. (Soweit nicht selbst erstellt). [vgl.]
- **Beispiel GmbH & BitLC:** Logos und Firmenidentität (Bereitgestellt durch den Ausbildungsbetrieb). [vgl.]
- **Eigene Darstellungen:** Alle Diagramme (UML, Architektur, Gantt) und Mockups wurden vom Verfasser selbst, mittels Mermaid.JS[13] und Google Nano Banana erstellt.

[13] [9] Mermaid Dokumentation

A.1 Verzeichnisstruktur

```
/
├── .editorconfig
├── .github
│   ├── dependabot.yml
│   └── workflows
│       └── main.yml
├── .gitignore
├── .postcssrc.json
├── README.md
├── angular.json
├── eslint.config.js
├── package.json
├── playwright.config.ts
├── docs/
├── public/
│   ├── assets
│   │   ├── data
│   │   └── images
│   ├── favicon.ico
│   └── favicon.svg
├── src/
│   ├── app
│   │   ├── app.config.ts
│   │   ├── app.routes.ts
│   │   ├── features/
│   │   ├── models/
│   │   ├── services/
│   │   ├── shared/
│   │   └── store/
│   ├── test-setup.ts
│   ├── main.ts
│   └── styles.css
├── tests/
│   ├── quiz.spec.ts
│   └── smoke.spec.ts
├── tsconfig.app.json
├── tsconfig.json
├── tsconfig.spec.json
└── vitest.config.ts
```

```
/docs
├── assets/
│   ├── UI/
│   │   └── UISelection.webp
│   ├── BITLC.png
│   ├── BeispielGMBH.png
│   ├── logo.webp
│   ├── UIMockUp.png
│   └── Ui.png
├── Abnahmeprotokoll.md .
├── BENUTZERHANDBUCH.md
├── ProjektAntragFCPP.pdf
└── README_TECHNICAL.md
```

```
/user/plugins/eol-manager/
├── assets/
├── classes/
├── data/
├── templates/
├── blueprints.yaml
├── eol-manager.php
└── README.md
```

```
/
├── src/
│   ├── app/
│   │   ├── core/
│   │   │   ├── base-api.service.ts
│   │   │   ├── index.ts
│   │   │   └── services/
│   │   │       └── ocean-data.service.ts
│   │   ├── models/
│   │   │   ├── index.ts
│   │   │   ├── ocean.model.ts
│   │   │   └── inhabitant.model.ts
│   │   ├── features/
│   │   ├── shared/
│   │   ├── store/
│   │   │   ...
│   │   ├── styles.css
│   │   ...
│   ├── tsconfig.json
│   ...
```

A.2. Quellcode und Dateien

Der vollständige und kommentierte Quellcode aller Projektdateien, Grafiken, Protokolle, Lasten-. und Pflichtenheft et cetera, befindet sich im beiliegenden digitalen Anhang (ZIP-Archiv).

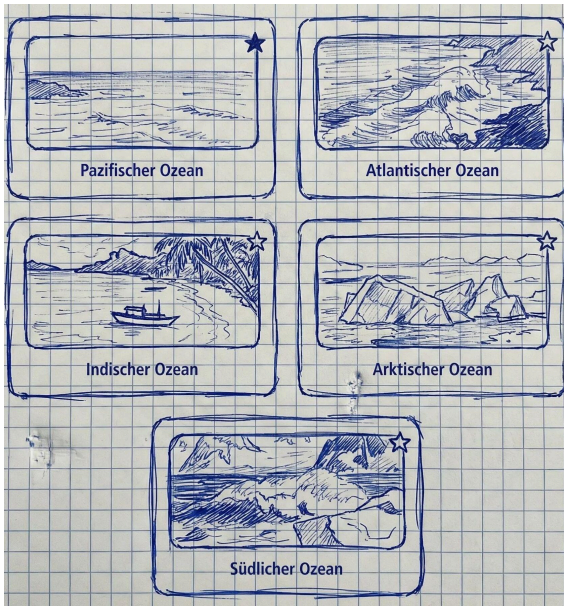
A.3. Mockups und Screenshots

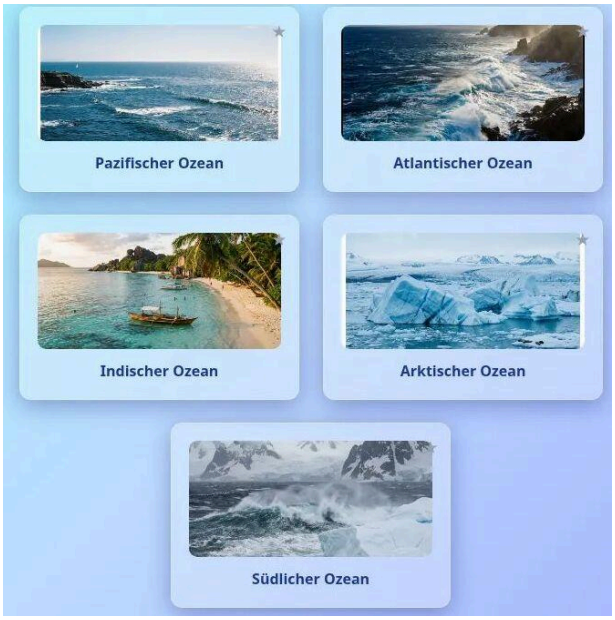
start:



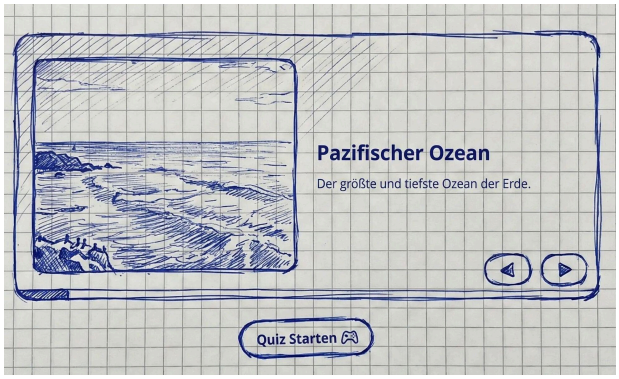


ocean-selection:



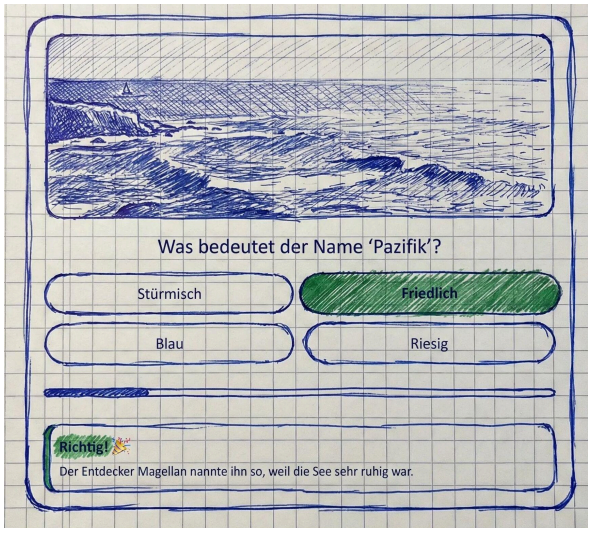


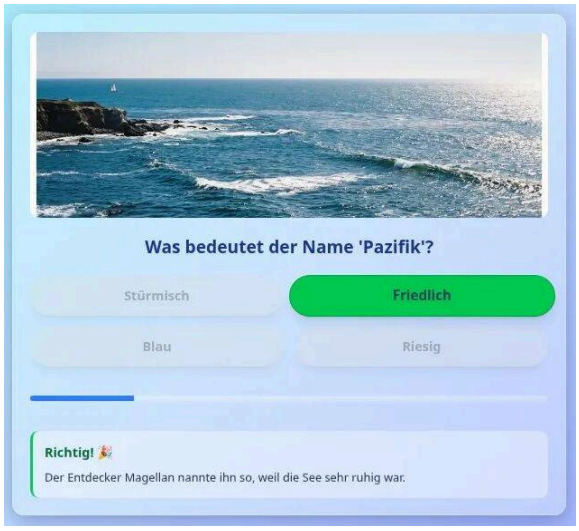
ocean-facts:



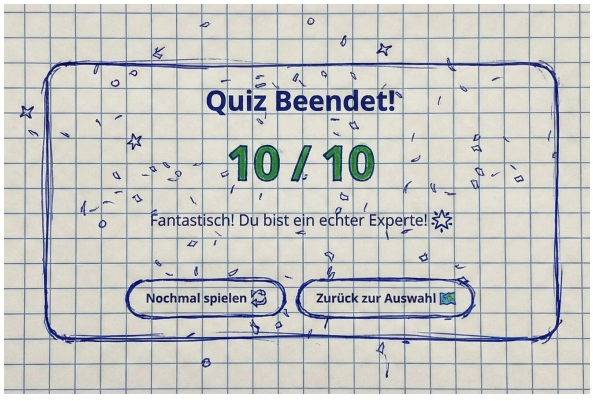


quiz:





quiz-results:





A.4 Erweiterung der FCPP Umsetzungs-Elemente

Für den initialen FCPP-Antrag siehe ProjektAntragFCPP.PDF

PE10 - Start Bildschirm

- PE10/UM11:AF10: Umsetzung des "Glas-Effekts" mittels Tailwind CSS Utility-Classes (`glass-card`, `backdrop-filter: blur(12px)`, `bg-white/25`).
- PE10/UM31:AF30: Implementierung der Animationen durch CSS Keyframes (`animate-fly-in`, `animate-pop-in`) definiert in der `styles.css`.
- PE10/UM51:AF50: Routing-Logik in der `start.component.ts` mittels Angular Router (`this.router.navigate(['/selection'])`).

PE20 - Auswahlbildschirm

- PE20/UM21:AF20: Dynamisches Rendering der Ozean-Karten unter Verwendung der Angular Control Flow Syntax (`@for`) basierend auf dem `store.oceans()` Signal.
- PE20/UM22:AF20: Visuelle Darstellung des Fortschritts (Stern-Icon) durch Prüfung von `store.isOceanCompleted(ocean.id)` mit bedingten CSS-Klassen (`text-yellow-400`).
- PE20/UM41:AF40: Implementierung des "Master Quiz" Buttons, der mittels `store.isMasterUnlocked()` Signal erst klickbar wird, wenn 5 Sterne gesammelt wurden (deaktivierter Zustand via `opacity-50`).
- PE20/UM42:AF40: Visuelle Fortschrittsanzeige als Balken unten im Button, berechnet aus der Anzahl der gelösten Ozeane (`store.completedOceans().length / 5`).

PE30 - Ozeanfakten

- PE30/UM41:AF40: Aggregation aller Daten (Basis-Info, Fakten-Array, Bewohner-Array) in ein einziges `computed` Signal (`slides`), für die einheitliche Navigation.
- PE30/UM51:AF50: Fallback-Logik für Bilder mittels `(error)="handleMissingImage($event)"`, wenn ein Pfad im JSON fehlerhaft ist.
- PE30/UM81:AF80: Logik-Sperre für den "Quiz Starten" Button: Ein `hasViewedAll` Signal schaltet den Button erst sichtbar (`opacity-100`), wenn der User einmal alle Slides durchgeklickt hat.

PE40 - Quiz Bildschirm

- PE40/UM31:AF30: Optimiertes Laden des Quiz-Bildes mittels `fetchpriority="high"` und `loading="eager"` für bessere Performance.
- PE40/UM41:AF40: Abruf der aktuellen Frage und Optionen reaktiv über `store.currentQuestion()` aus dem `QuizStore`.
- PE40/UM42:AF40: Implementierung eines 5-Sekunden-Timers (nur im Master-Modus), der über ein Signal (`timeLeft`) gesteuert wird und bei Ablauf als falsche Antwort gewertet wird.
- PE40/UM51:AF50: Styling der Antwort-Buttons basierend auf dem Zustand (`selectedOption` Signal): Zuweisung von Klassen wie `!bg-green-500` oder `grayscale` für nicht gewählte Optionen.

PE50 - Quiz Richtig / PE60 - Quiz Falsch (Zusammengefasst in QuizComponent Logik)

- PE50/UM41:AF40: Visuelles Feedback durch Setzen des `showFeedback` Signals, welches eine Box mit grüner (`border-green-500`) oder roter Umrandung einblendet.
- PE50/UM42:AF40: Verzögerter Übergang zur nächsten Frage (`setTimeout` 1500ms), damit der User den Trivia-Text lesen kann.
- PE60/UM41:AF40: Bei falscher Antwort wird die gewählte Option rot markiert, aber die korrekte Antwort grün hervorgehoben (`getOptionClass` Logik), um den Lerneffekt zu sichern.

PE70 - Auswertung

- PE70/UM31:AF30: Unterscheidung in der Template-Logik (`@if (store.masterMode())`) zwischen normaler Ergebnisanzeige und der "Urkunde" für das Master-Quiz.
- PE70/UM32:AF30: Integration der `canvas-confetti` Bibliothek in der `ngOnInit` [14] Methode, die bei 100% Score (`isSuccess()`) Konfetti regnen lässt.
- PE70/UM41:AF40: Automatische Speicherung des Fortschritts im `localStorage` durch einen Angular `effect` im `QuizStore`, sobald ein Quiz erfolgreich beendet wurde.

[14] [6] *canvas-confetti Documentation*

A.5 Code-Beispiele

```
import { Injectable, computed, inject, signal, effect } from '@angular/core';
import { OceanDataService } from '../services/ocean.data.service';
import { Ocean, QuizQuestion } from '../models/ocean.model';

/**
 * Zentraler Service für die Verwaltung der Anwendungslogik und des Spielzustands.
 *
 * Dieser Service nutzt das "Service with Signals"-Pattern, um den State (Zustand)
 * der Anwendung reaktiv zu verwalten. Er kümmert sich um das Laden der Daten,
 * die Steuerung des Quiz-Ablaufs (Start, Antwort, Ende) sowie die Persistierung
 * des Lernfortschritts im LocalStorage.
 */
@Injectable({
  providedIn: 'root'
})
export class QuizStore {
  // Injektion des OceanDataService zum Abruf der statischen JSON-Daten
  private OceanDataService = inject(OceanDataService);
  // Konstante für den Schlüssel, unter dem der Fortschritt im LocalStorage
gespeichert wird
  private readonly STORAGE_KEY = 'eol_progress';
  // --- Interner Zustand (Private Writable Signals) ---
  // Diese Signale sind privat, um den Zustand nur über definierte Methoden änderbar
zu machen.

  /** Liste aller geladenen Ozeane mit ihren Daten. */
  private _oceans = signal<Ocean[]>([]);
  /** ID des aktuell ausgewählten Ozeans (oder 'master' für das Master-Quiz). */
  private _currentOceanId = signal<string | null>(null);
  /** Index der aktuellen Frage im Fragen-Array (0-basiert). */
  private _currentQuestionIndex = signal<number>(0);
  /** Aktuelle Punktzahl des Spielers im laufenden Quiz. */
  private _score = signal<number>(0);
  /** Status-Flag: Gibt an, ob gerade ein Quiz aktiv läuft. */
  private _isQuizActive = signal<boolean>(false);

  /** Status-Flag: Gibt an, ob das Quiz beendet wurde (für die Ergebnisanzeige). */
  private _isQuizFinished = signal<boolean>(false);
  /** Liste der IDs aller Ozeane, die erfolgreich (100% richtig) abgeschlossen
wurden. */
  private _completedOceans = signal<string[]>([]);
  /** Status-Flag: Gibt an, ob sich der Spieler im "Master-Quiz"-Modus befindet. */
  private _masterMode = signal<boolean>(false);
  // --- Öffentliche Schnittstelle (Public Read-Only Signals) ---
  // Komponenten können diese Signale konsumieren, aber nicht direkt verändern.
  readonly oceans = this._oceans.asReadonly();
  readonly currentOceanId = this._currentOceanId.asReadonly();
  readonly currentQuestionIndex = this._currentQuestionIndex.asReadonly();
  readonly score = this._score.asReadonly();
  readonly isQuizActive = this._isQuizActive.asReadonly();
```

```
  readonly isQuizFinished = this._isQuizFinished.asReadonly();
  readonly completedOceans = this._completedOceans.asReadonly();
  readonly masterMode = this._masterMode.asReadonly();
  // --- Abgeleitete Zustände (Computed Signals) ---
  // Diese Werte werden automatisch neu berechnet, wenn sich ihre Abhängigkeiten
ändern.
  /**
   * Liefert das Objekt des aktuell ausgewählten Ozeans basierend auf der ID.
   */
  readonly currentOcean = computed(() =>
    this._oceans().find(o => o.id === this._currentOceanId())
  );
  /**
   * Liefert die aktuelle Quizfrage basierend auf dem gewählten Ozean und dem
Frage-Index.
   */
  readonly currentQuestion = computed(() => {
    const ocean = this.currentOcean();
    return ocean?.quiz[this._currentQuestionIndex()] || null;
  });
  /**
   * Berechnet die Gesamtanzahl der Fragen im aktuellen Quiz.
   */
  readonly totalQuestions = computed(() =>
    this.currentOcean()?.quiz.length || 0
  );
  /**
   * Berechnet den aktuellen Fortschritt im Quiz in Prozent (für die Progress-Bar).
   */
  readonly progress = computed(() => {
    const total = this.totalQuestions();
    if (total === 0) return 0;
    return (this._currentQuestionIndex() / total) * 100;
  });
  /**
   * Prüft, ob das Master-Quiz freigeschaltet ist (alle 5 Ozeane abgeschlossen).
   */
  readonly isMasterUnlocked = computed(() =>
    this._completedOceans().length >= 5
  );
  constructor() {
    // Beim Initialisieren des Services wird versucht, einen gespeicherten
Fortschritt
    // aus dem LocalStorage des Browsers zu laden.
    const savedProgress = localStorage.getItem(this.STORAGE_KEY);
    if (savedProgress) {
      try {
        this._completedOceans.set(JSON.parse(savedProgress));
      } catch (e) {
        console.error('Fehler beim Laden des Fortschritts:', e);
      }
    }
    // Ein Angular Effect überwacht das Signal `_completedOceans`.
```

```
    // Sobald sich der Wert ändert (z.B. neuer Stern gewonnen), wird der neue Zustand
    // automatisch im LocalStorage persistiert.
    effect(() => {
      const progress = this._completedOceans();
      localStorage.setItem(this.STORAGE_KEY, JSON.stringify(progress));
    });
  }
  /**
   * Prüft, ob ein spezifischer Ozean bereits erfolgreich abgeschlossen wurde.
   * @param oceanId Die ID des zu prüfenden Ozeans.
   * @returns True, wenn der Ozean in der Liste der abgeschlossenen Ozeane enthalten
ist.
   */
  isOceanCompleted(oceanId: string): boolean {
    return this._completedOceans().includes(oceanId);
  }
  // --- Methoden zur Zustandsänderung ---
  /**
   * Lädt die Ozean-Daten vom Backend (JSON-Datei).
   * Die Antwortmöglichkeiten jeder Frage werden hierbei zufällig gemischt,
   * um Abwechslung beim Spielen zu gewährleisten.
   */
  loadOceans() {
    this.OceanDataService.getOceans().subscribe((oceans) => {
      const shuffledOceans = oceans.map(ocean => ({
        ...ocean,
        quiz: ocean.quiz.map(q => ({
          ...q,
          // Kopieren und zufälliges Sortieren der Antwortmöglichkeiten
          options: [...q.options].sort(() => Math.random() - 0.5)
        }))
      }));
      this._oceans.set(shuffledOceans);
    });
  }
  /**
   * Wählt einen Ozean aus und bereitet den Quiz-Zustand vor.
   * @param oceanId Die ID des ausgewählten Ozeans.
   */
  selectOcean(oceanId: string) {
    this._currentOceanId.set(oceanId);
    this.resetQuizState();
  }
  /**
   * Startet den normalen Quiz-Modus.
   * Setzt Punktzahl und Frage-Index zurück.
   */
  startQuiz() {
    this._isQuizActive.set(true);
    this._score.set(0);
    this._currentQuestionIndex.set(0);
    this._isQuizFinished.set(false);
    this._masterMode.set(false);
```

```
  }
  /**
   * Initialisiert und startet das "Master-Quiz".
   * Logik:
   * 1. Sammelt alle Fragen aus allen Ozeanen (außer einem evtl. alten
Master-Eintrag).
   * 2. Mischt alle Fragen zufällig.
   * 3. Erstellt ein temporäres "Master-Ozean"-Objekt, das diese gemischten Fragen
enthält.
   */
  startMasterQuiz() {
    // Sammeln aller Fragen (FlatMap)
    const allQuestions = this._oceans()
      .filter(o => o.id !== 'master')
      .flatMap(o => o.quiz);
    // Zufälliges Mischen aller Fragen
    const shuffledQuestions = [...allQuestions].sort(() => Math.random() - 0.5);
    // Erstellen eines virtuellen Ozean-Objekts für das Master-Quiz
    const masterOcean: Ocean = {
      id: 'master',
      name: 'Ultimatives Quiz',
      color: 'bg-purple-600',
      oceanimage: '/assets/images/pacific.png', // Platzhalter-Bild oder spezifisches
Master-Bild
      description: 'Das ultimative Quiz gegen die Zeit! 🏆',
      facts: [],
      inhabitants: [],
      quiz: shuffledQuestions
    };
    // Temporäres Hinzufügen des Master-Ozeans zum State
    this._oceans.update(oceans => {
      const others = oceans.filter(o => o.id !== 'master');
      return [...others, masterOcean];
    });
    // Setzen des Zustands für den Master-Modus
    this._currentOceanId.set('master');
    this._currentQuestionIndex.set(0);
    this._score.set(0);
    this._isQuizActive.set(true);
    this._isQuizFinished.set(false);
    this._masterMode.set(true);
  }
  /**
   * Wertet die Antwort des Benutzers aus.
   * Erhöht den Punktestand, wenn die Antwort korrekt ist.
   * @param answer Die vom Benutzer gewählte Antwort als String.
   */
  answerQuestion(answer: string) {
    const question = this.currentQuestion();
    if (question && answer === question.answer) {
      this._score.update(s => s + 1);
    }
  }
```

```
  /**
   * Schaltet zur nächsten Frage weiter.
   * Wenn die letzte Frage erreicht wurde, wird das Quiz beendet.
   */
  nextQuestion() {
    const nextIndex = this._currentQuestionIndex() + 1;
    const total = this.totalQuestions();

    if (nextIndex >= total) {
      this.finishQuiz();
    } else {
      this._currentQuestionIndex.set(nextIndex);
    }
  }
  /**
   * Interne Methode zum Abschließen des Quiz.
   * Prüft im normalen Modus, ob alle Fragen richtig beantwortet wurden (Perfekte
Punktzahl),
   * und speichert in diesem Fall den Ozean als "abgeschlossen" (Sternvergabe).
   */
  private finishQuiz() {
    this._isQuizActive.set(false);
    this._isQuizFinished.set(true);
    // Fortschritt speichern: Nur im normalen Modus und bei 100% richtiger Antworten
    if (!this._masterMode() && this._score() === this.totalQuestions()) {
      const currentId = this._currentOceanId();
      // Verhindert Duplikate in der Liste der abgeschlossenen Ozeane
      if (currentId && !this._completedOceans().includes(currentId)) {
        this._completedOceans.update(list => [...list, currentId]);
      }
    }
  }
  /**
   * Startet das aktuelle Quiz neu, ohne die Auswahl zu verlassen.
   */
  restartQuiz() {
    this._score.set(0);
    this._currentQuestionIndex.set(0);
    this._isQuizActive.set(true);
    this._isQuizFinished.set(false);
  }
  /**
   * Beendet das Quiz komplett und setzt die Auswahl zurück (z.B. bei Klick auf
"Zurück").
   */
  exitQuiz() {
    this._isQuizActive.set(false);
    this._isQuizFinished.set(false);
    this._currentOceanId.set(null);
    this._masterMode.set(false);
  }
  /**
   * Hilfsmethode, um interne Zähler und Flags zurückzusetzen.
```

```
   */
  private resetQuizState() {
    this._currentQuestionIndex.set(0);
    this._score.set(0);
    this._isQuizActive.set(false);
    this._isQuizFinished.set(false);
    this._masterMode.set(false);
  }}
```

CI/CD Pipeline Konfiguration (.github/workflows/main.yml)

```yaml
name: Angular CI

on:
  push:
    branches: ['main', 'master']
  pull_request:
    branches: ['main', 'master']

jobs:
  build:
    name: Build & Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'

      - name: Install Dependencies
        run: npm ci

      - name: Linting
        run: npm run lint
        continue-on-error: true # Don't fail build on lint warnings for now

      - name: Run Unit Tests
        run: npm run test:coverage

      - name: Install Playwright Browsers
        run: npx playwright install --with-deps

      - name: Run E2E Tests
        run: npm run test:e2e

      - name: Production Build
        run: npm run build -- --configuration production

      - name: Archive Build Artifacts
        uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/
```

Modernes Styling mit Tailwind 4 @utility (Auszug aus styles.css)

```
/* src/styles.css */
@import "tailwindcss";

:root {
    --glass-bg: rgba(255, 255, 255, 0.25);
}

/* Definition einer Custom Utility in Tailwind 4 */
@utility glass-card {
    background: var(--glass-bg);
    backdrop-filter: blur(12px);
    border: 1px solid rgba(255, 255, 255, 0.3);
    /* Mixin von Tailwind Utilities */
    @apply rounded-2xl p-6 shadow-lg;
}
```

Generischer Basis-Service (Auszug aus `base-api.service.ts`)

```
// src/app/core/base-api.service.ts
import { HttpClient } from '@angular/common/http';
import { inject } from '@angular/core';
import { Observable } from 'rxjs';

/**
 * Abstrakte Basisklasse für API-Services.
 * Implementiert generische CRUD-Methoden, um Code-Duplizierung zu vermeiden.
 * @template T - Der Typ der Entität (z.B. Ocean).
 */
export abstract class BaseApiService<T> {
  protected http = inject(HttpClient);

  // Muss von der ableitenden Klasse definiert werden
  protected abstract resourceUrl: string;

  getAll(): Observable<T[]> {
    return this.http.get<T[]>(this.resourceUrl);
  }

  getById(id: number | string): Observable<T> {
    return this.http.get<T>(`${this.resourceUrl}/${id}`);
  }
}
```

A.6 Benutzerhandbuch

Version: 1.0 | Stand: 09.12.2025 | Zielgruppe: Schüler (7-9 Jahre) & Lehrkräfte

1. Einführung Willkommen bei Earth Ocean Learning! Diese interaktive Web-App hilft Kindern, die fünf Weltmeere spielerisch zu entdecken. Die Anwendung ist intuitiv gestaltet und erfordert keine Installation.

2. Systemvoraussetzungen

- Geräte: Tablet (empfohlen), PC, Laptop oder Smartphone.
- Browser: Aktuelle Version von Chrome, Firefox, Edge oder Safari.
- Internet: Für das erste Laden erforderlich. Danach sind Inhalte teilweise offline verfügbar (PWA/Cache).

3. Erste Schritte

1. Öffnen Sie den Browser und rufen Sie die App-URL auf.
2. Sie sehen den Startbildschirm mit dem Logo.
3. Klicken Sie auf den großen Button "Los geht's", um das Hauptmenü zu betreten.

4. Die Ozean-Auswahl (Hauptmenü) Auf der Übersichtskarte werden die 5 Weltmeere als Kacheln dargestellt.

- Navigation: Tippen Sie auf eine Karte, um in den Lernmodus dieses Ozeans zu gelangen.
- Fortschritt: Oben rechts auf jeder Karte befindet sich ein Stern. Dieser wird gold, sobald das Quiz für diesen Ozean bestanden wurde.
- Master-Quiz: Am unteren Rand befindet sich das "Ultimative Quiz". Dieser Button bleibt gesperrt (grau), bis alle 5 Sterne gesammelt wurden.

5. Lernen & Entdecken

- Fakten: Kurze, kindgerechte Texte erklären Besonderheiten (z. B. "Der tiefste Ozean").
- Bewohner: Eine Bildergalerie zeigt typische Tiere des jeweiligen Ozeans.
- Steuerung: Nutzen Sie die Pfeiltasten, um durch die Fakten zu blättern.
- Quiz starten: Erst nachdem alle Fakten angesehen wurden, wird der Button "Quiz Starten" aktiv.

6. Das Quiz spielen

- Ablauf: Es werden 10 Fragen gestellt.
- Feedback:
  - Grün: Richtig!
  - Rot: Falsch (Die richtige Antwort wird grün umrandet).
- Ergebnis: Bei 10 von 10 Punkten erhalten Sie den Stern für diesen Ozean.

7. Das Ultimative Quiz (Experten-Modus) Sobald alle 5 Sterne gesammelt sind, wird der Button "Ultimatives Quiz Starten" lila und aktiv.

- Besonderheit: Hier herrscht Zeitdruck! Pro Frage haben Sie nur 5 Sekunden Zeit.
- Belohnung: Bei Erfolg erhalten Sie die digitale Urkunde als "Ozean-Experte".

8. Problembehandlung

- Fortschritt weg? Bitte nutzen Sie nicht den "Inkognito/Privat"-Modus, da der Fortschritt lokal gespeichert wird.
- Kein Ton? Die Version 1.0 enthält bewusst noch keine Soundeffekte (geplant für V2.0).

A.7 Abnahmeprotokoll

Projekt: Earth Ocean Learning (EOL) App

Auftragnehmer: Beispiel GmbH (Prüfling: Tobias Boyke)

Auftraggeber: NGO "OzeanSchutz" (Vertreten durch: Dr. Uwe Umwelt)

Datum der Abnahme: 09.12.2025

1. Gegenstand der Abnahme

Gegenstand ist die funktionstüchtige Single Page Application (SPA) zur Vermittlung von Ozean-Wissen inkl. Quellcode und Dokumentation.

2. Durchgeführte Funktionstests

Folgende kritische Funktionen wurden geprüft und abgenommen (basierend auf Testprotokoll):

ID	Funktion / Anforderung	Erwartetes Ergebnis	Status	Bemerkung
TF01	App-Start	Startbildschirm lädt, "Los geht's" klickbar	OK	Lädt in < 1s
TF02	Ozean-Auswahl	Navigation zu Fakten-Seite funktioniert	OK	-
TF03	Quiz-Logik	Korrekte Auswertung (Grün/Rot) bei Klick	OK	Feedback sofort sichtbar
TF04	Fortschritt (Persistence)	Speicherung der Sterne im LocalStorage	OK	Bleibt nach Reload erhalten
TF05	Master-Quiz Unlock	Button gesperrt bis 5 Sterne erreicht	OK	Visuell ausgegraut
TF06	Responsivität	Layout passt sich Tablet/Desktop an	OK	Getestet auf Pixel Tablet

**

3. Offene Mängel

Keine kritischen Mängel bekannt.

Hinweis: Sound-Effekte und Mehrsprachigkeit sind laut Lastenheft erst für Version 2.0 geplant.

4. Erklärung

Die Software erfüllt die im FCPP (Lastenheft) definierten Anforderungen. Die Abnahme wird hiermit ohne Vorbehalte erklärt.

Unterschrift Auftragnehmer Unterschrift Auftraggeber

(Tobias Boyke) (Dr. Uwe Umwelt)

A.8 Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe angefertigt und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Neuss, 08.12.2025 (Unterschrift Tobias Boyke)

A.9 Realer Zeitaufwand & Reflexion

Während der Prüfungszeitraum auf 70 Stunden begrenzt ist, erforderte die Einarbeitung in die neuesten Technologien (Angular 21 RC, Signals-Architektur) sowie die Erstellung der umfangreichen Assets und des Backends einen realen Arbeitsaufwand, der weit darüber hinausging.

Um das Projekt in der vorliegenden Qualität fertigzustellen, wurde über den gesamten Zeitraum von 23 Tagen (17.11. – 09.12.2025) in einem intensiven Modus gearbeitet.

Berechnungsgrundlage:

- Arbeitsmodus: 6 Tage pro Woche
- Tägliche Arbeitszeit: ca. ~10 Real Stunden
- Zeitraum: 17.11.2025 bis 09.12.2025

Detaillierte Aufstellung:

Woche	Zeitraum	Arbeitstage	Stunden (ca.)	Fokus
Woche 1	**17.11. – 22.11.**	**7 Tage**	**60 h**	**Einarbeitung Angular 21, Setup, Design-Entwürfe, Core-Logic, Diagramme, GitHub**
Woche 2	**24.11. – 29.11.**	**7 Tage**	**65 h**	**Implementierung Core-Logik, Store-Pattern, Backend-Plugin, Doku, GitHub, CSS**
Woche 3	**01.12. – 06.12.**	**7 Tage**	**70 h**	**UI-Feinschliff, Animationen, Responsive Design, Testing, Refactoring, Vitest, Playwright, Doku**
Woche 4	**08.12. – 09.12.**	**2 Tage**	**10 h**	**Abschlussdokumentation, Bugfixing, Grav**
Gesamt		**23 Tage**	**205 h**	**Real investierte Projektzeit**

Dieser Wert (**205 h**) verdeutlicht das Engagement, das in die Entwicklung der "Earth Ocean Learning"-App geflossen ist, und übersteigt die prüfungsrelevante Zeitmessung (70 h) **um den Faktor 2,9.**

Basierend auf dem tatsächlich geleisteten Zeitaufwand von 205 Stunden (siehe A.9) ergibt sich eine angepasste Kosten- und Amortisationsrechnung. Diese spiegelt den wahren Investitionswert wider, der nötig war, um die Software in diesem Reifegrad ("Polished Product") zu erstellen.

Nachkalkulation der Real-Kosten: Anstelle der prüfungsrelevanten 70 Stunden werden hier die vollen 205 Stunden angesetzt.

- Personalkosten (Real): 205 h × 9,00 €/h = 1.845,00 €
- Sachmittel (Pauschale): 150,00 €
- **Gesamtkosten (Real): 1.995,00 €**

Amortisationsrechnung (Real): Die jährliche Einsparung durch den Wegfall der Druckkosten bleibt konstant (vgl. Kap. 6.3).

- Jährliche Einsparung (Cash Flow): 1.740,00 €
- Investitionskosten (Real): 1.995,00 €

$$Amortisationsdauer = \frac{1.995,00 €}{1.740,00 €/Jahr} \approx 1,15 \text{ Jahre}$$

Fazit: Selbst unter Berücksichtigung des massiven, privaten Mehraufwands bleibt das Projekt hochwirtschaftlich. Die Investition amortisiert sich bereits nach **1 Jahr und 2 Monaten**. Dies bestätigt, dass die Entscheidung zur Eigenentwicklung auch bei realistischer Betrachtung der Entwicklungszeit für die NGO finanziell vorteilhaft ist.

Während der gesamten Phase, wurden Dokumentationen und das Herunterbrechen des Inhalts mittels Gemini 3.0Pro Thinking LLM, in nachvollziehbaren bits, zu einer großen Unterstützung.

Die einzelnen Hauptaufgaben des Projekts waren anspruchsvoll und jede für sich eine Challenge, angefangen bei der Entwurfsphase (welche mich vor die Frage stellte “Was zum heck ist ein FCPP?”),

Planung (mittels Github & KanBan Google Notes und Chrome Tabs, Mermaid),

Das Implementieren des Frontends (BEM vs OOCSS vs SMACSS vs Atomic Design???, Will ich auf externe Bibs setzen oder lieber @angular/core-Funktionen? Ignoriere ich nun die fehlenden Unittests und linter, und konzentriere ich mich auf das wesentliche oder drücke ich auf der npm Paketseite von eslint und vitest mal wieder “F5”?).

Die Herausforderung des Backends (welches ich wie ein sturer Junge unbedingt nativ in grav laufen lassen wollte und deswegen noch mit rest.api und php arbeiten konnte!).

Bis hin zur Dokumentation, welche für mich zwar straightforward war, aber tausende edits hatte, bis ich zufrieden war.

Alles in allem war es eine lehrreiche Erfahrung, bei der ich vieles, was ich bereits kannte, umsetzen konnte, aber auch neue Methoden und Technologien einsetzen konnte.

Vielen Dank Herr Ehlen.

Unten Finden sie ein Rezept von Gemini ;)

A.10 Guten Appetit

Deftiges Entrecôte (Well Done) mit Gouda-Butter-Schmelze und Ofenkartoffeln

Als Beilage eignen sich hier hervorragend **knusprige Ofenkartoffeln (Wedges)**, da sie stabil genug sind, um die reichhaltige Butter-Käse-Sauce aufzunehmen.

Zutaten

Für das Fleisch:

- 300 g Entrecôte (Zimmertemperatur)
- 150 g Gouda (gerieben oder in Scheiben)
- 100 g Butter
- Salz & Pfeffer
- 1 Knoblauchzehe, angedrückt

Für die Beilage (Ofenkartoffeln):

- 400–500 g festkochende Kartoffeln
- 2 EL Pflanzenöl
- Grobes Meersalz, Paprikapulver, getrockneter Rosmarin

Zubereitung

1. Beilage vorbereiten (Starten Sie hiermit):

1. Den Backofen auf **200 °C Ober-/Unterhitze** (oder 180 °C Umluft) vorheizen.
2. Die Kartoffeln waschen und (mit Schale) in Spalten (Wedges) schneiden.
3. In einer Schüssel mit dem Öl, Salz, Paprikapulver und Rosmarin vermengen.
4. Auf einem Backblech verteilen und ca. **30–35 Minuten** backen, bis sie goldbraun und knusprig sind.

2. Fleisch braten (ca. 15 Min. bevor die Kartoffeln fertig sind):

1. Eine Pfanne stark erhitzen. Das gesalzene Entrecôte hineinlegen und von beiden Seiten je **3 Minuten** scharf anbraten, bis eine kräftige Kruste entsteht.
2. Die Hitze auf mittlere bis niedrige Stufe reduzieren.
3. Die **100 g Butter** und die Knoblauchzehe in die Pfanne geben.
4. Das Steak nun etwa **8–12 Minuten** in der Butter garziehen lassen ("Well Done"). Wichtig: Übergießen Sie das Fleisch ständig mit einem Löffel mit der flüssigen Butter (Arrosieren).
    - *Test:* Das Fleisch muss sich auf Druck sehr fest anfühlen (wie ein angespannter Handballen) und darf nicht federnd nachgeben.

3. Überbacken & Anrichten:

1. Wenn das Fleisch durchgebraten ist, die Hitze auf das Minimum reduzieren.
2. Den **150 g Gouda** direkt auf das Fleisch und in die umgebende Butter geben.
3. Deckel auflegen und warten, bis der Käse komplett zerlaufen ist.
4. Die fertigen Kartoffelspalten auf einen Teller geben.
5. Das Steak daneben platzieren und die gesamte Mischung aus geschmolzenem Käse und Butter über das Fleisch und die Kartoffeln gießen.
6. Alles frisch pfeffern.